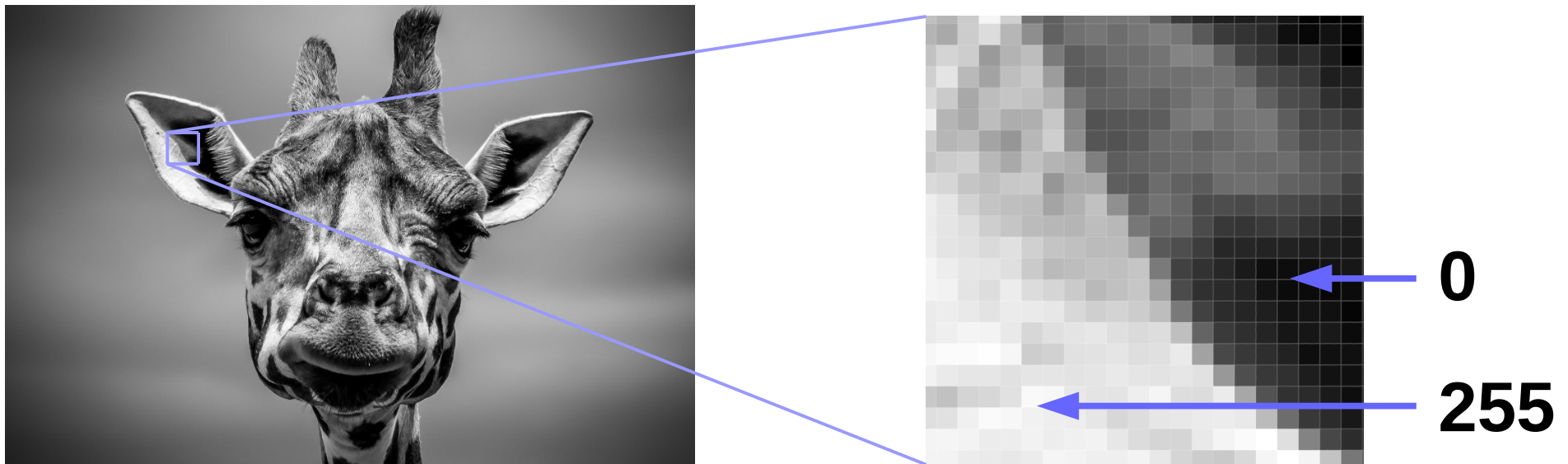


Лекция 5: Применение CNN

Images

- Grayscale image is a matrix of pixels [H x W]
 - Pixels = **p**icture **e**lements
- Each pixel stores number [0,255] for brightness



Images

- RGB image is a 3d array $[H \times W \times 3]$ or $[3 \times H \times W]$
 - Each pixel stores **R**ed, **G**reen & **B**lue color values $[0, 255]$

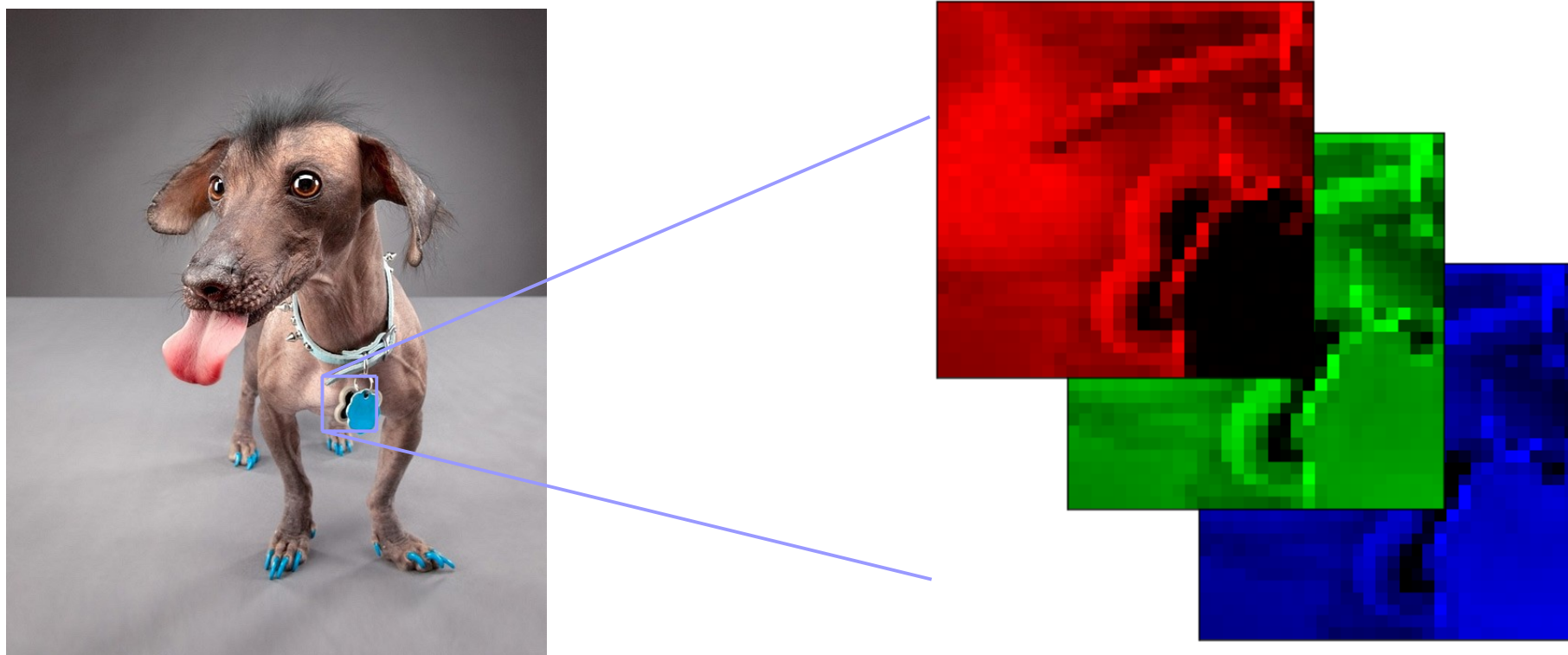
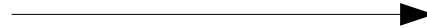
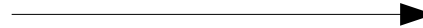


Image recognition



“Dog”

Image recognition



“Gray wall”

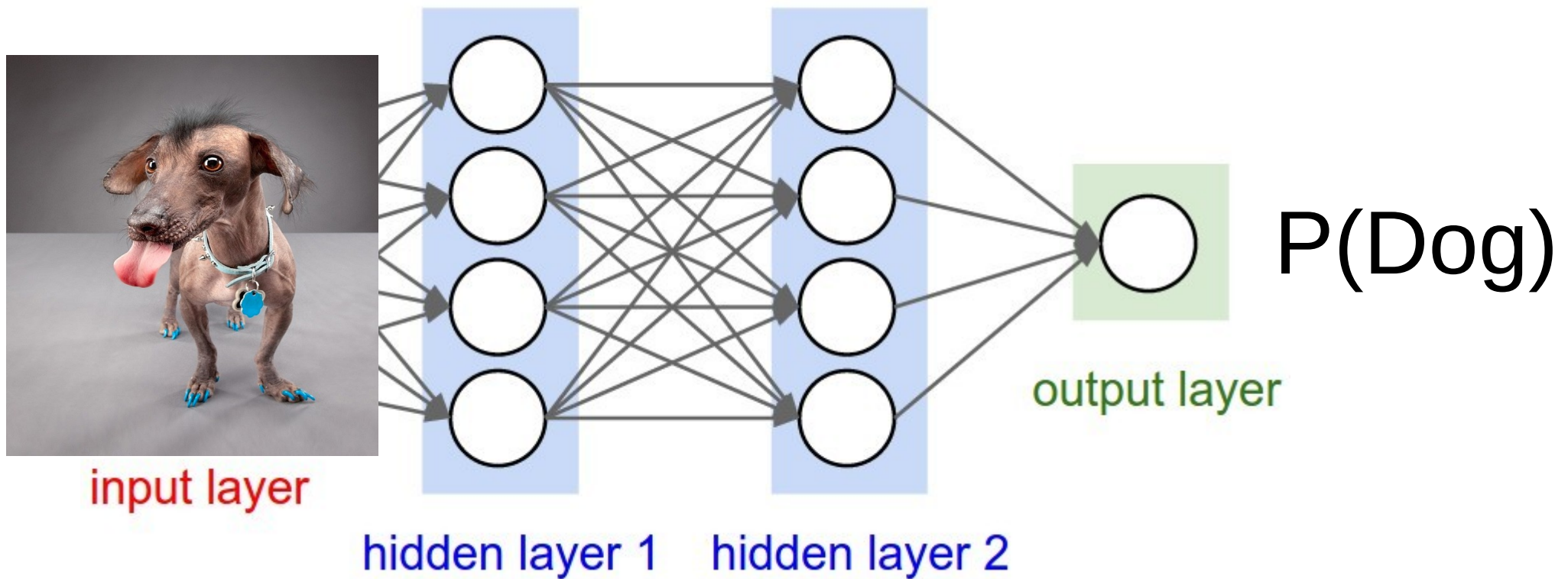
“Dog tongue”

“Dog”

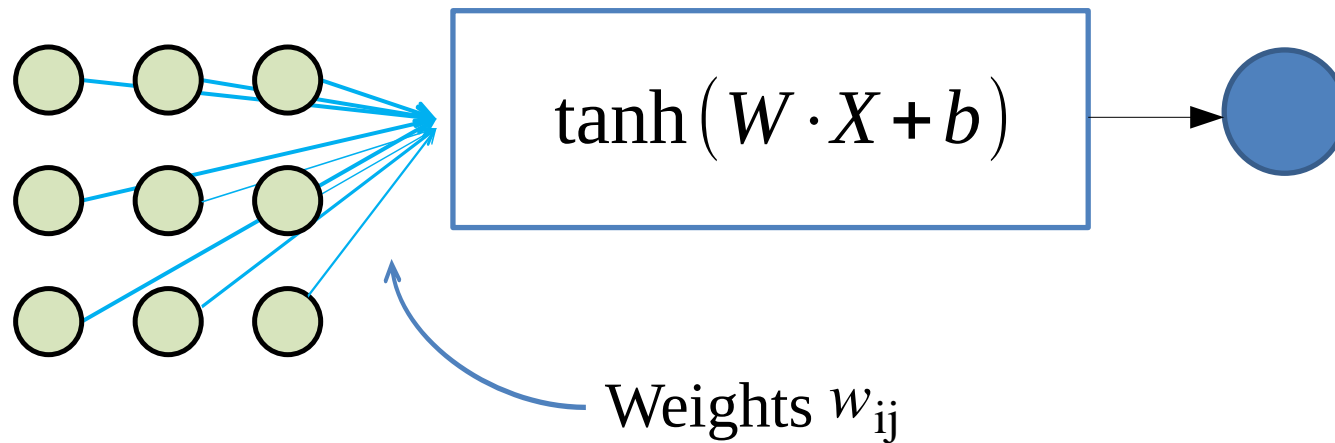
<a particular kind
of dog>

“Animal sadism”

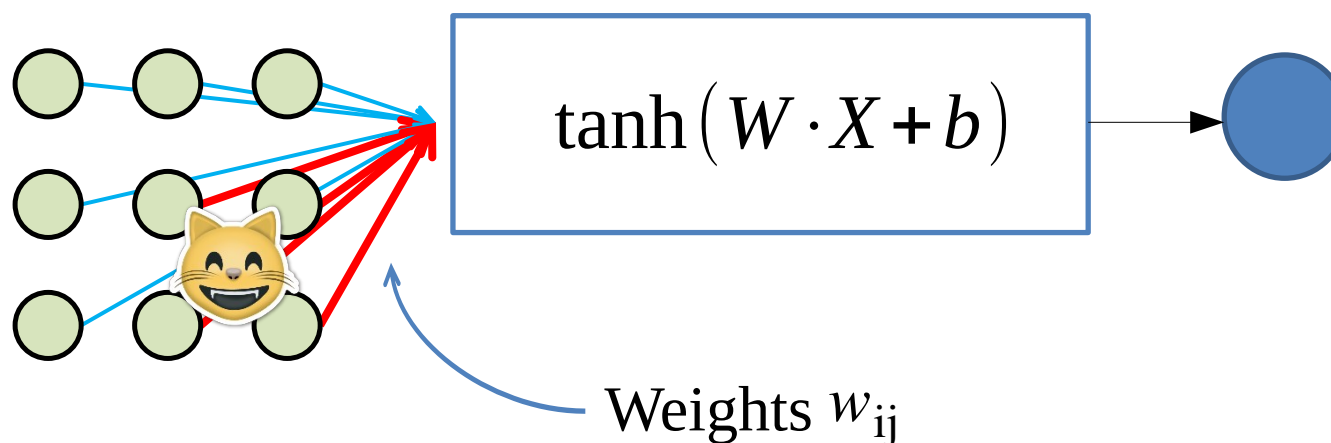
NN approach



Problem with images

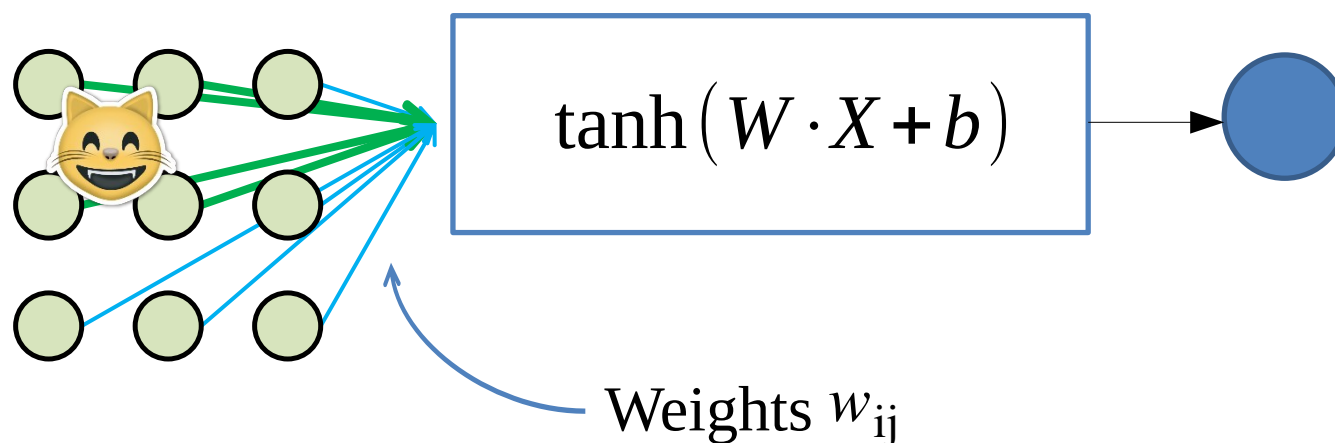


Problem with images



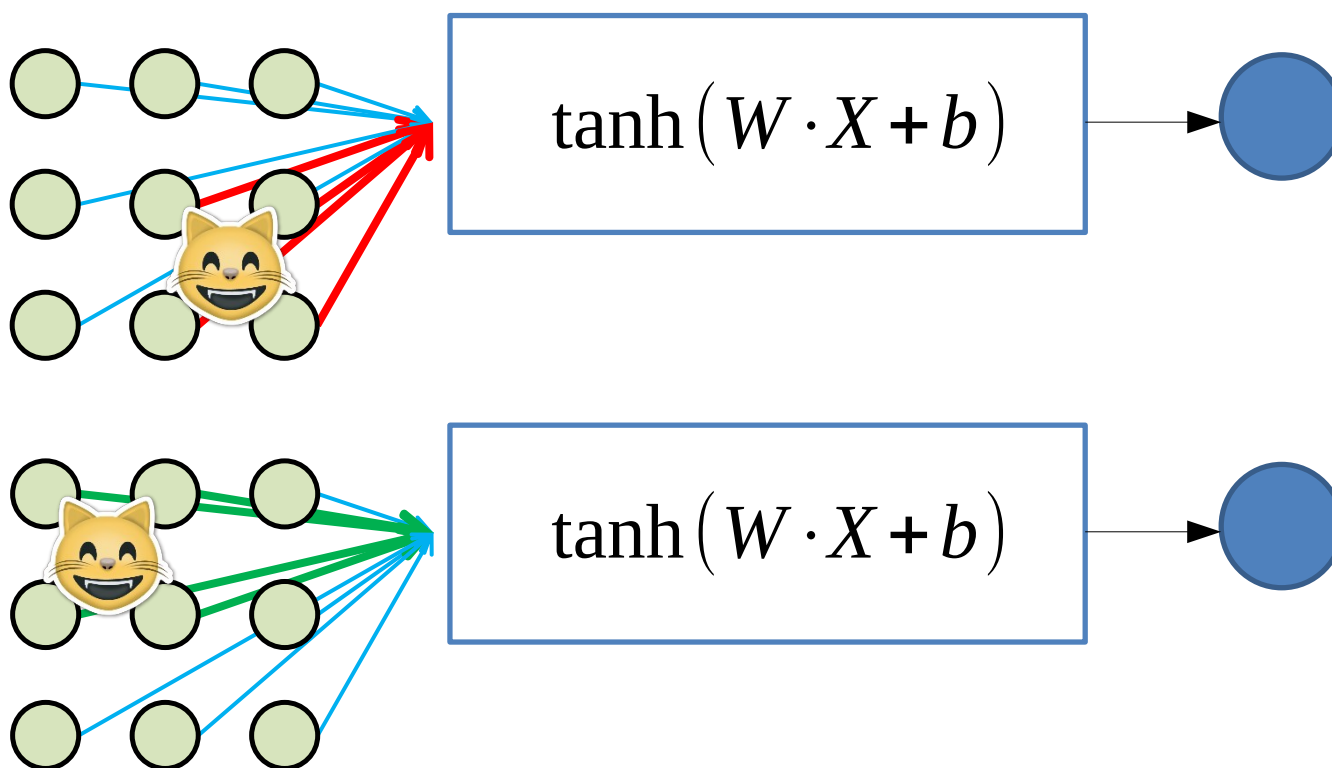
On this object, you will train **red** weights to react on cat face

Problem with images



On this object, you will train **green** weights to react on cat face

Problem with images



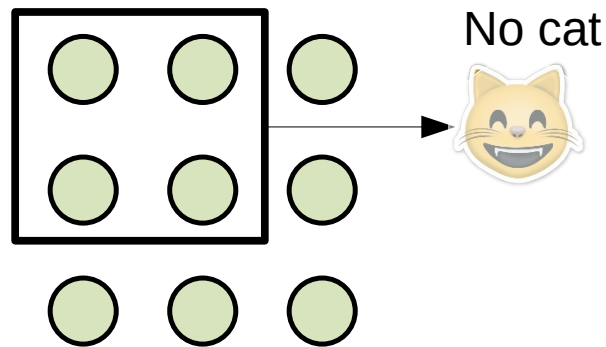
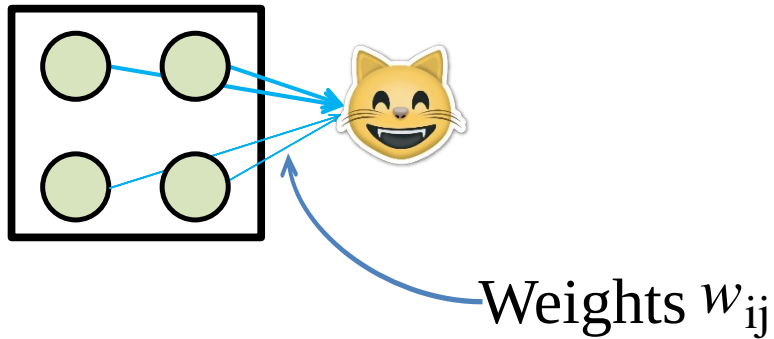
You network will have to learn those two cases separately!
Worst case: one neuron per position.

Problem

Idea: force all these “cat face” features
to use **exactly same weights**,
shifting weight matrix each time.

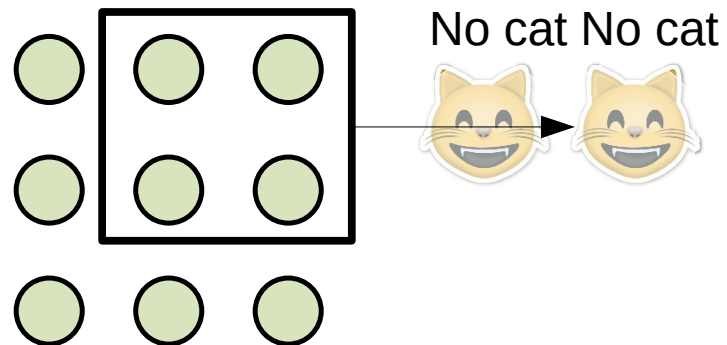
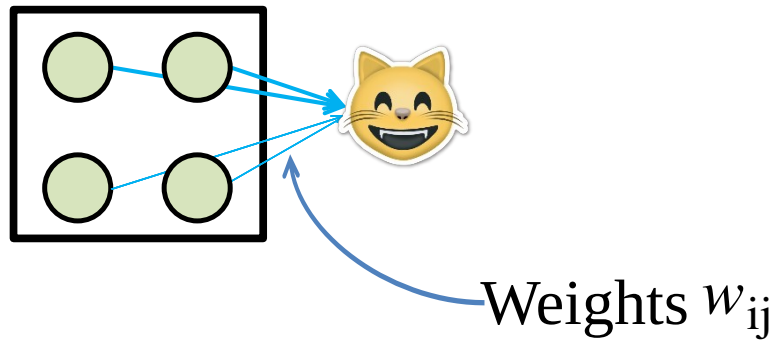
Same features for each spot

Portable cat detector pro!



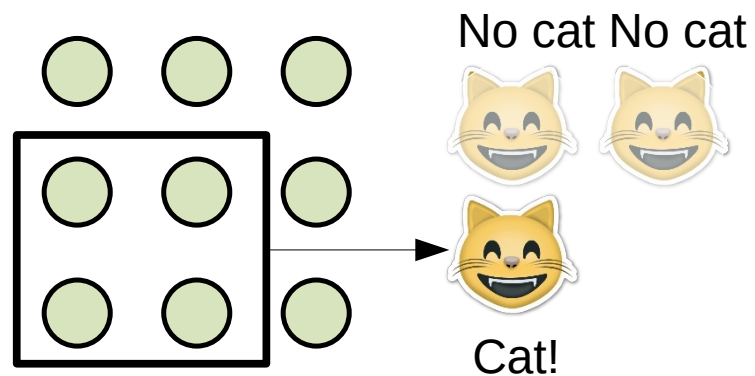
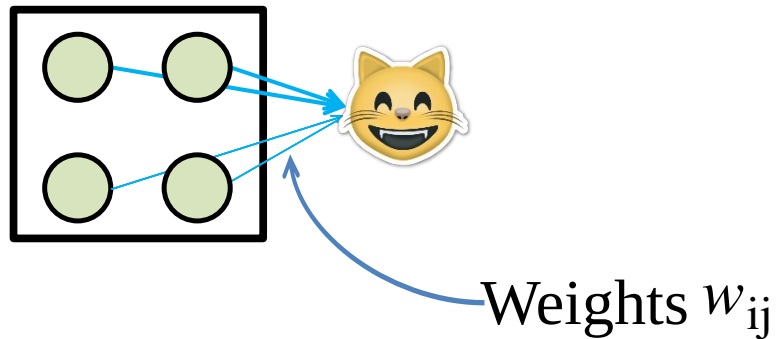
Same features for each spot

Portable cat detector pro!



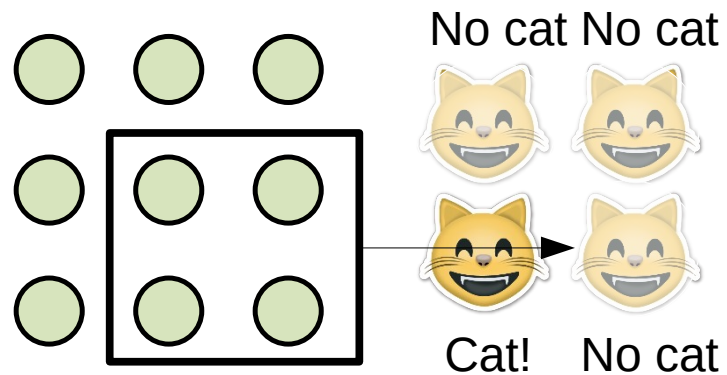
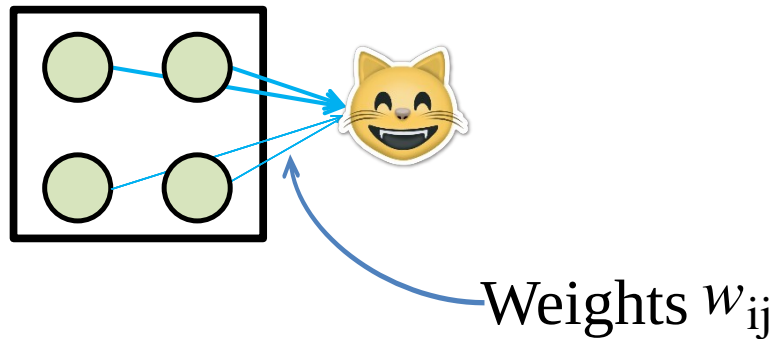
Same features for each spot

Portable cat detector pro!



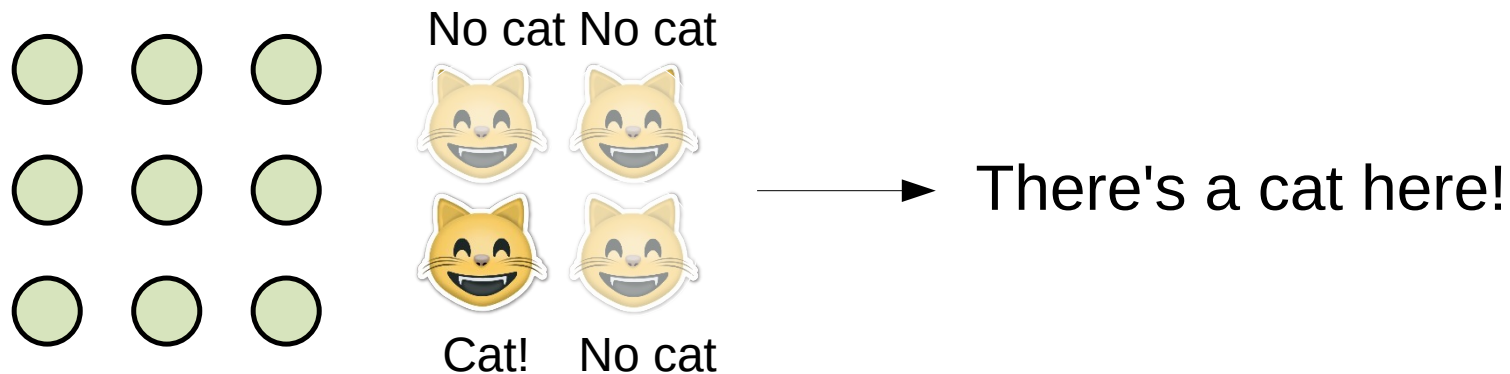
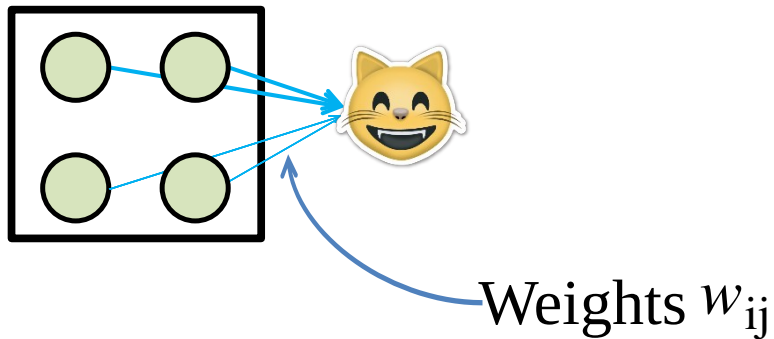
Same features for each spot

Portable cat detector pro!



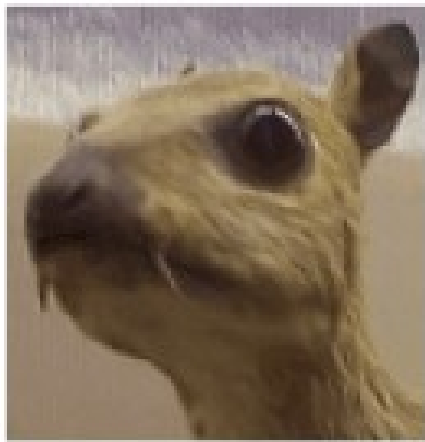
Same features for each spot

Portable cat detector pro!



Convolution

Input image



Convolution
Kernel

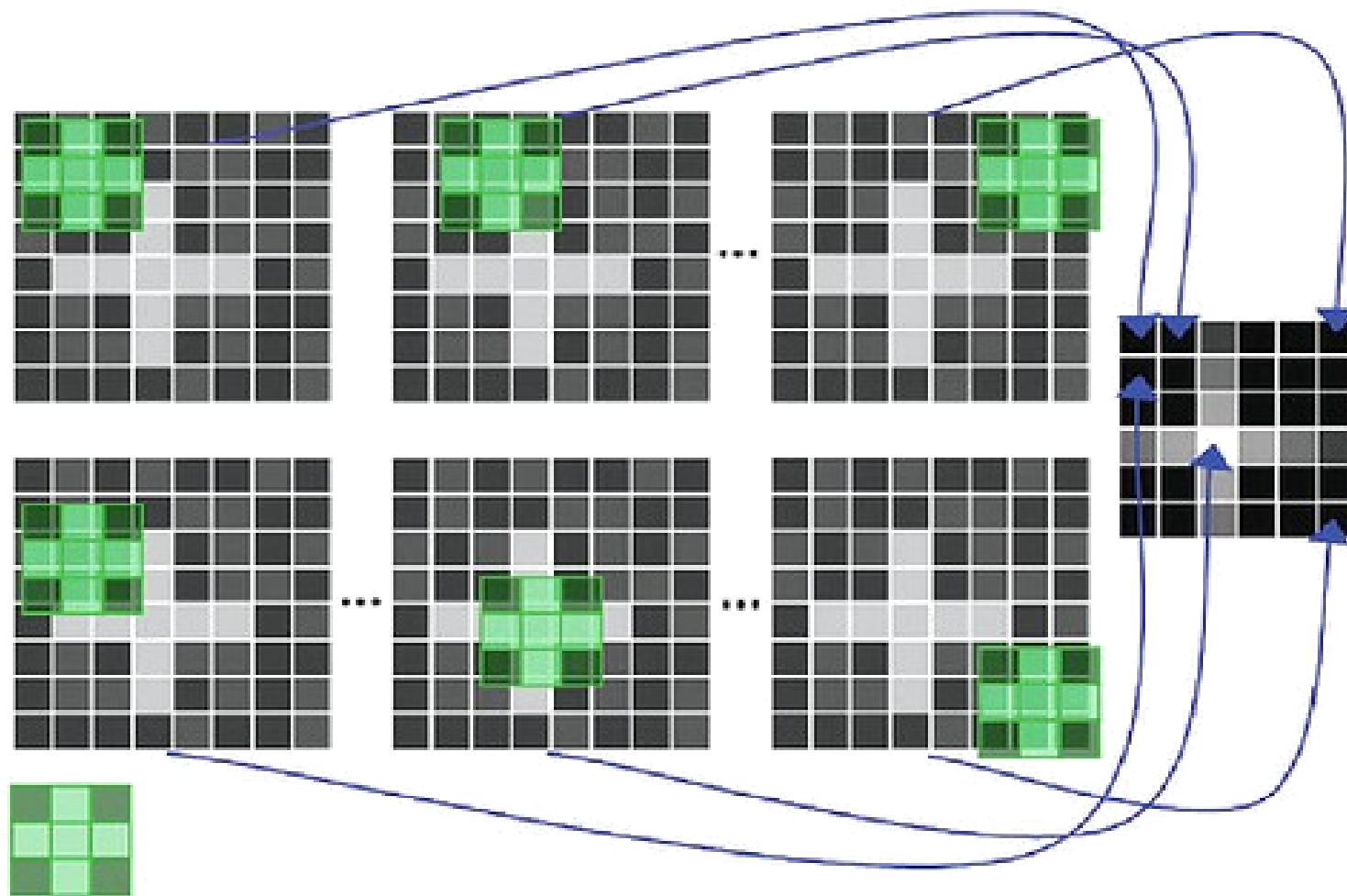
$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Feature map



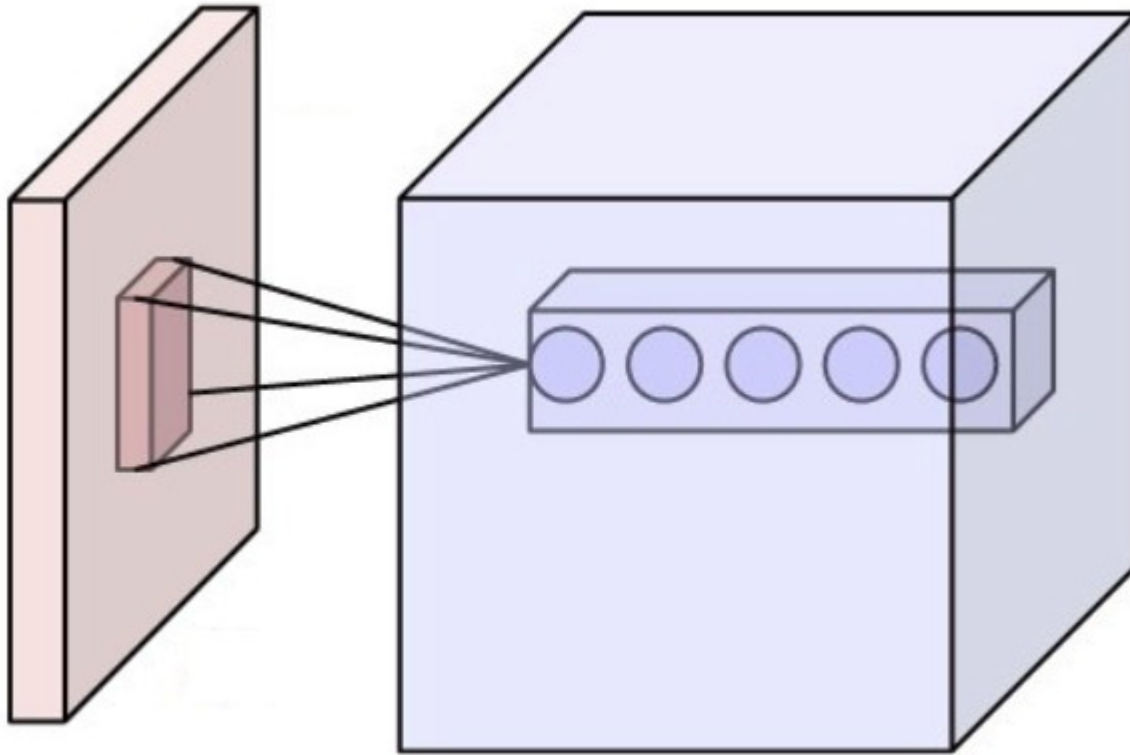
Intuition: how **edge-like** is this square?

Convolution



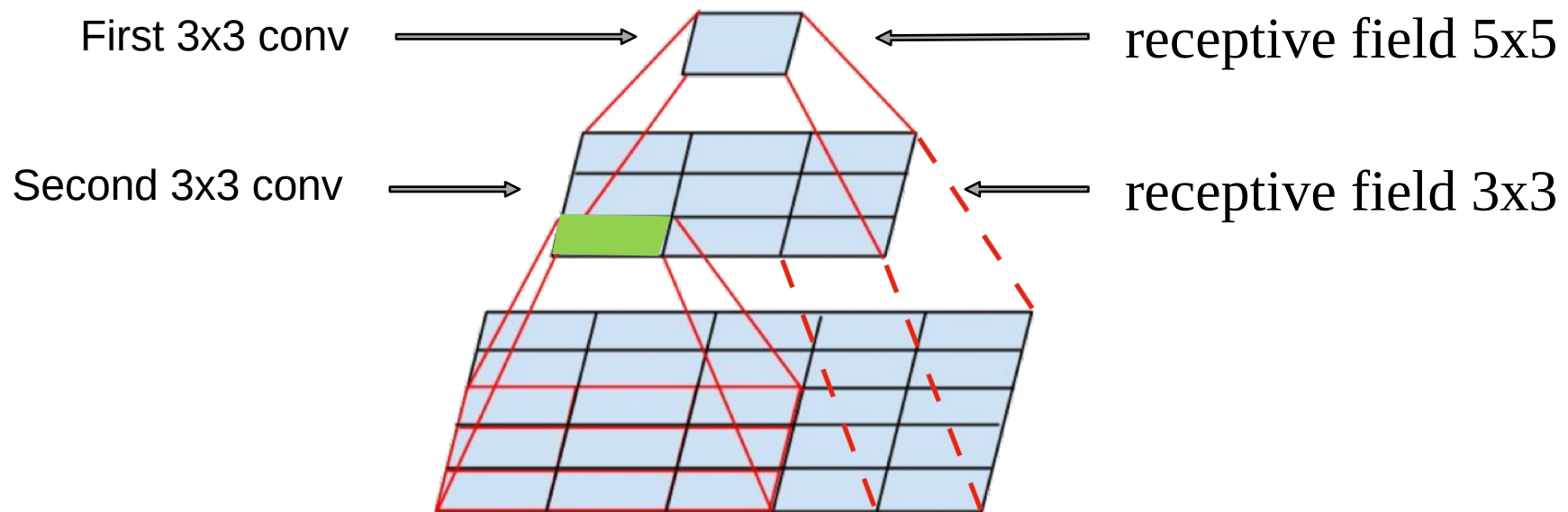
apply same filter to all patches

Convolution



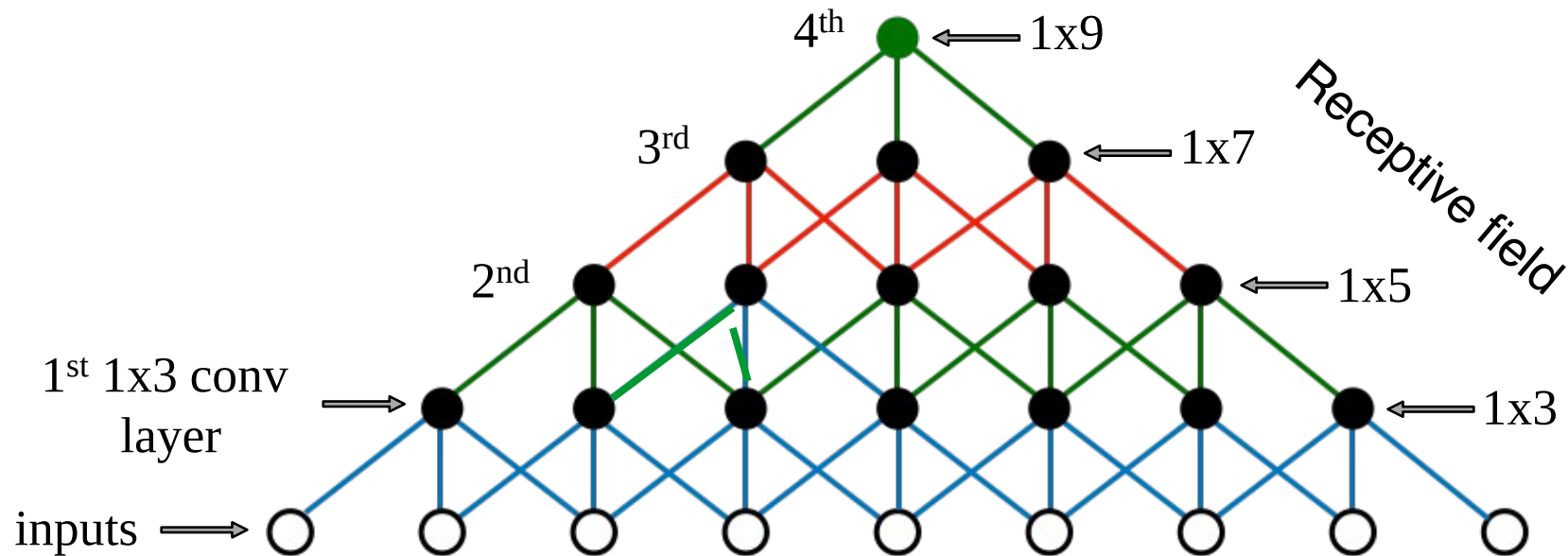
Intuition: how cat-like is this square?

Receptive field



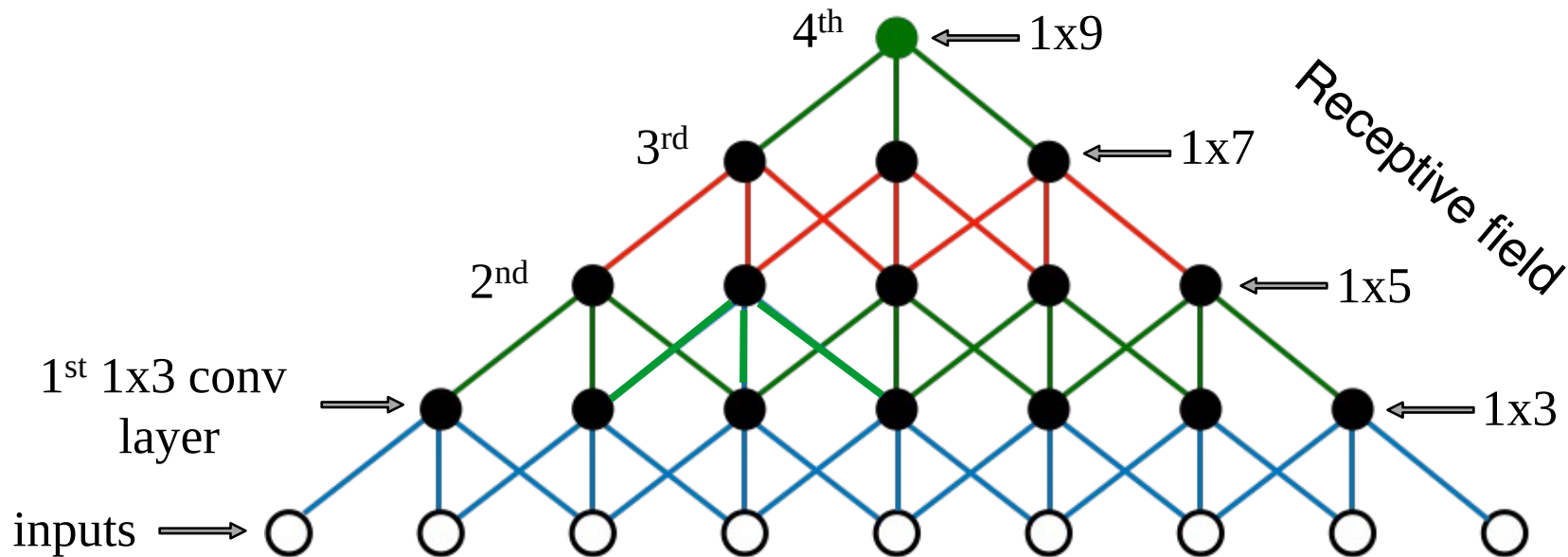
We can recognize larger objects by stacking several small convolutions!

Receptive field



Q: how many 3x3 convolutions we should use to recognize a 100x100px cat

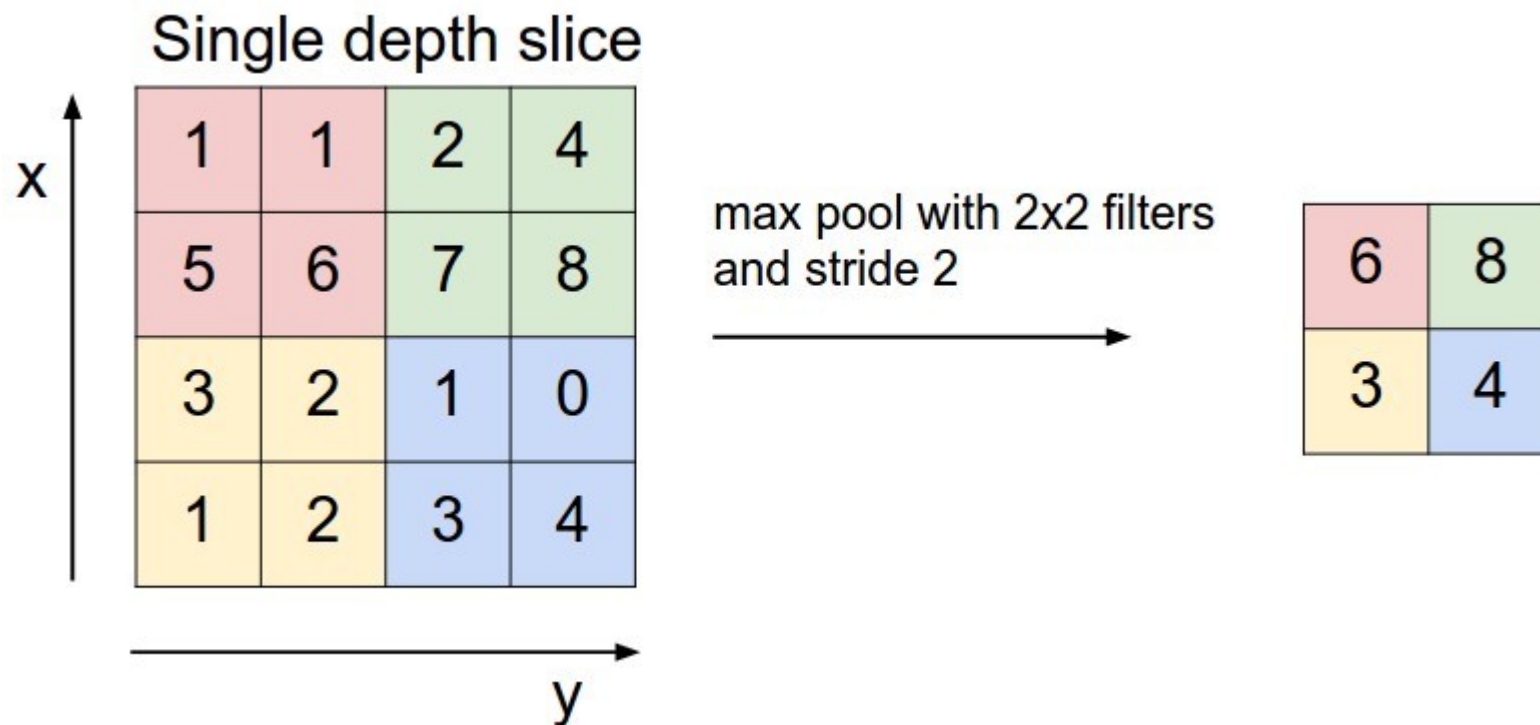
Receptive field



Q: how many 3x3 convolutions we should use to recognize a 100x100px cat

A: around 50... we need to increase receptive field faster!

Pooling



Intuition: What is the max cat-likelihood over this area?

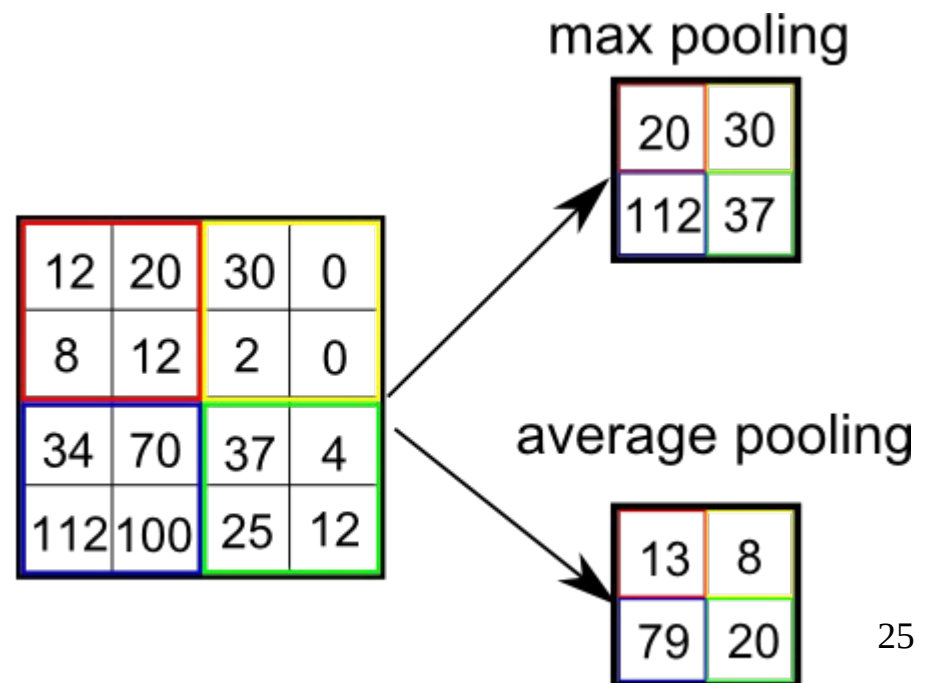
Pooling

Motivation:

- Reduce layer size by a factor
- Make NN less sensitive to small image shifts

Popular types:

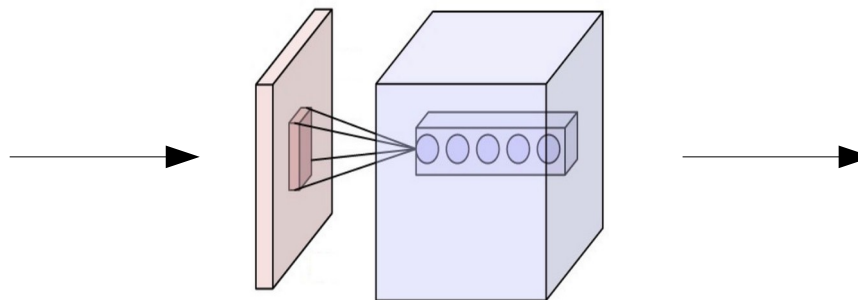
- Max
- Mean(average)



Convolution



Image : 3 (RGB) x 100 px x 100 px

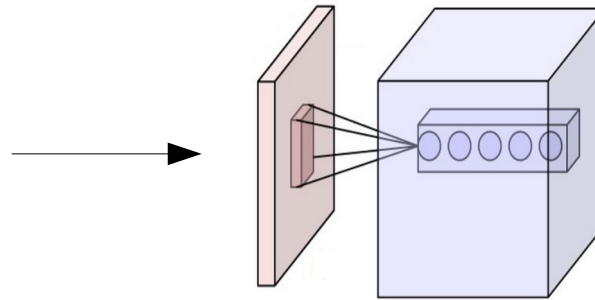


Filters: 100x(3x5x5)

Convolution



Image : 3 (RGB) x 100 px x 100 px



Filters: 100x(3x5x5)

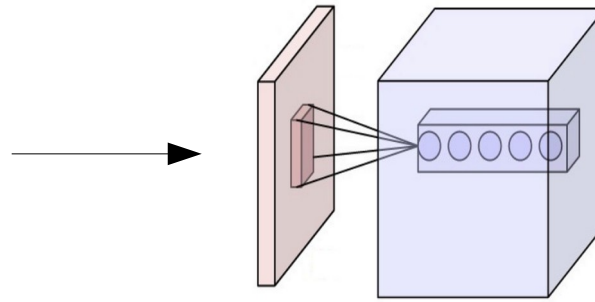
100x96x96
~10⁶

Somewhat too many!

Convolution



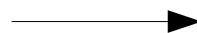
Image : 3 (RGB) x 100 px x 100 px



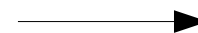
Filters: 100x(3x5x5)

100x96x96
~10⁶

100x96x96



pool
3x4

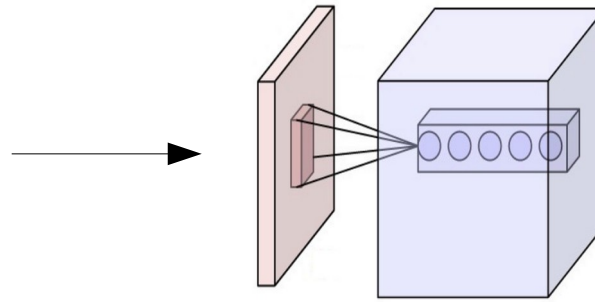


???

Convolution



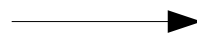
Image : 3 (RGB) x 100 px x 100 px



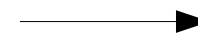
Filters: 100x(3x5x5)

100x96x96
~10⁶

100x96x96



pool
3x4



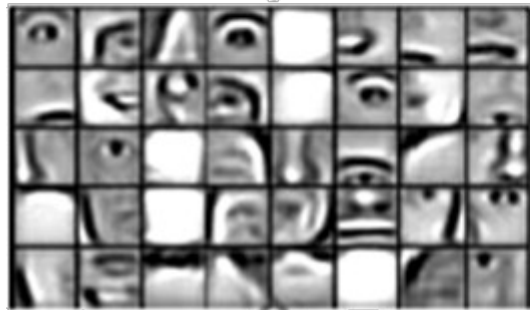
100x32x32

~10⁵

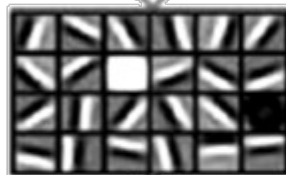


Discrete Choices

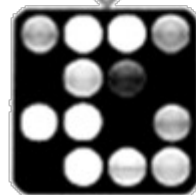
⋮



Layer 2 Features

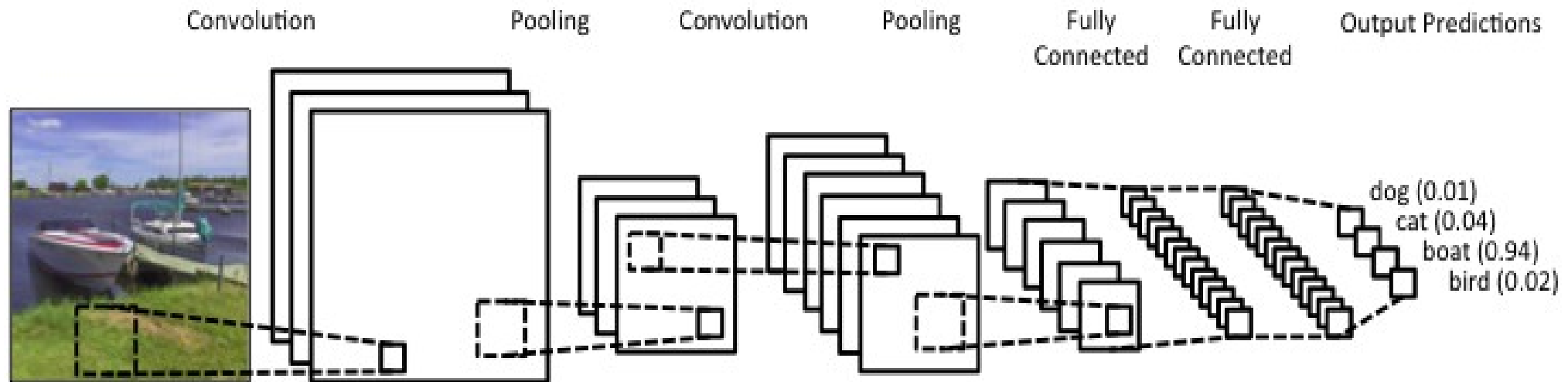


Layer 1 Features



Original Data

Convolutional NNs



Problem with large networks

What you sign for if you stack 1000 layers:

- `MemoryError(0x...)`
- Gradients can vanish
- Gradients can explode
- Activations can vanish
- Activations can explode
- Overfits like crazy

Recap: Batch normalization

Idea:

- We normalize activation of a hidden layer
(zero mean unit variance)

$$h_i = \frac{h_i - \mu_i}{\sqrt{\sigma_i^2}}$$

i stands for i-th neuron

- Update μ_i, σ_i^2 with moving average while training

$$\mu_i := \alpha \cdot \text{mean}_{\text{batch}} + (1 - \alpha) \cdot \mu_i$$

$$\sigma_i^2 := \alpha \cdot \text{variance}_{\text{batch}} + (1 - \alpha) \cdot \sigma_i^2$$

Architectures: residual network

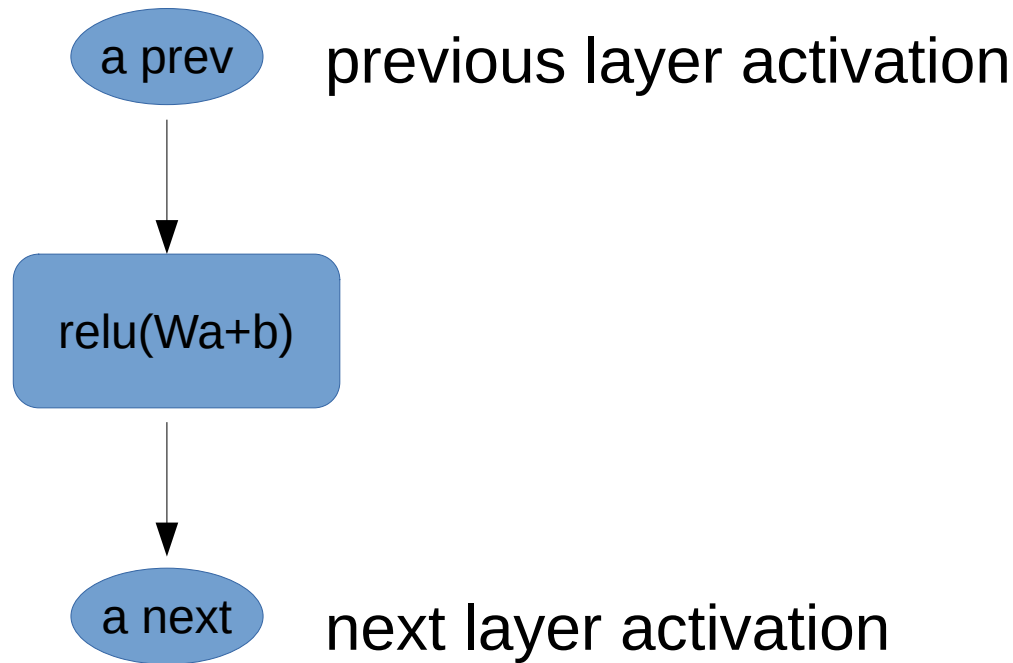
Problem: very deep networks are hard to train

Gradients w.r.t. first layers are volative

Architectures: residual network

Idea: let's create a shortcut for gradients

Normal layer

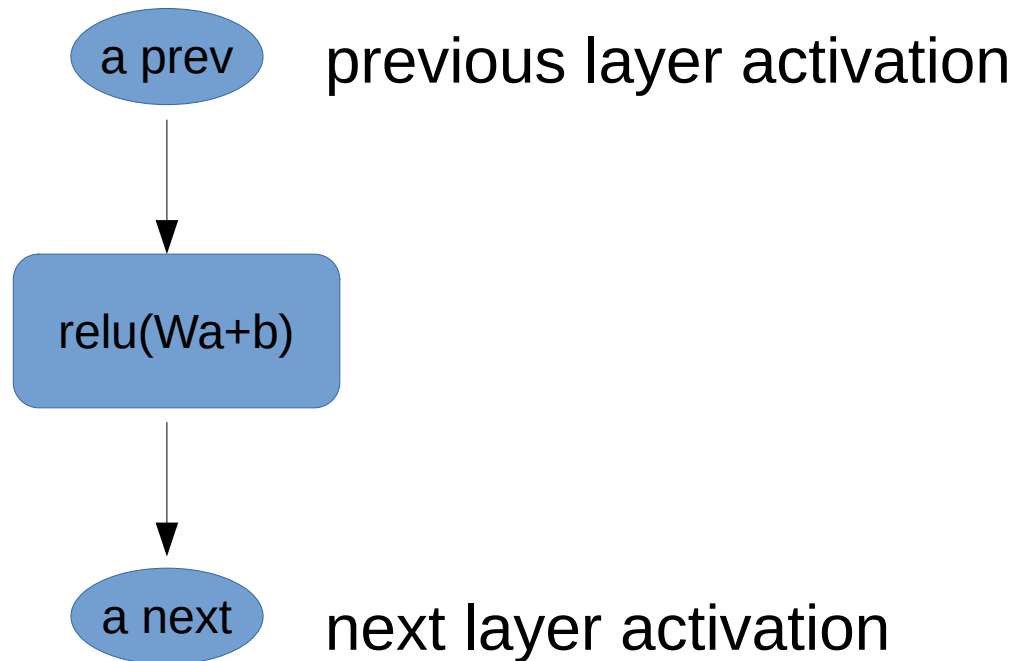


$$f_{w,b}(x) = \text{relu}(W \cdot a + b)$$

Architectures: residual network

Idea: let's create a shortcut for gradients

Normal layer



$$f_{w,b}(x) = \text{relu}(W \cdot a + b)$$

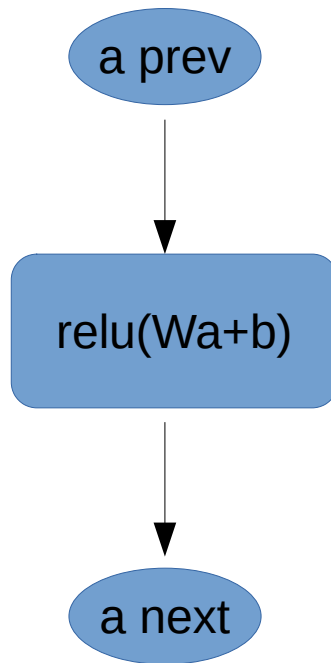
$$\nabla f_{w,b}(x) = \underline{\nabla \text{relu}(W \cdot a + b)}$$

Gradients can vanish if $\text{relu} < 0$

Architectures: residual network

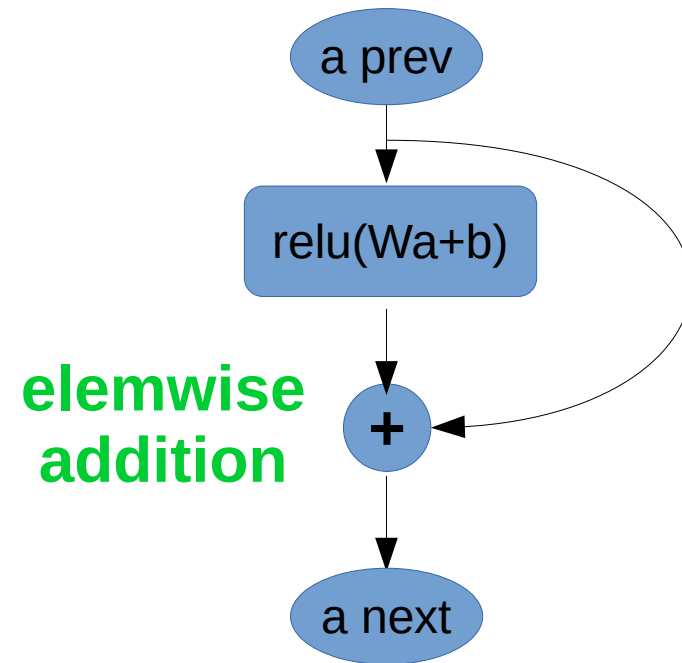
Idea: let's create a shortcut for gradients

Normal layer



$$f_{w,b}(x) = \text{relu}(W \cdot a + b)$$

Residual layer

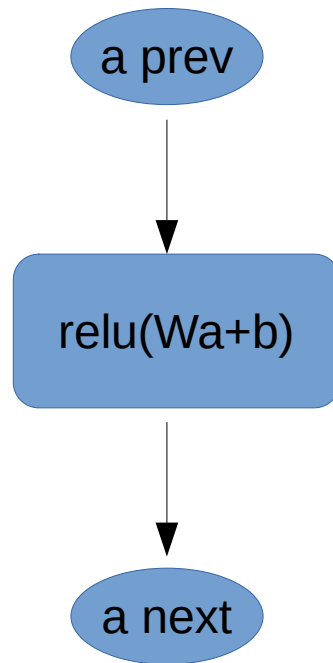


$$f_{w,b}(x) = \text{relu}(W \cdot a + b) + X$$

Architectures: residual network

Idea: let's create a shortcut for gradients

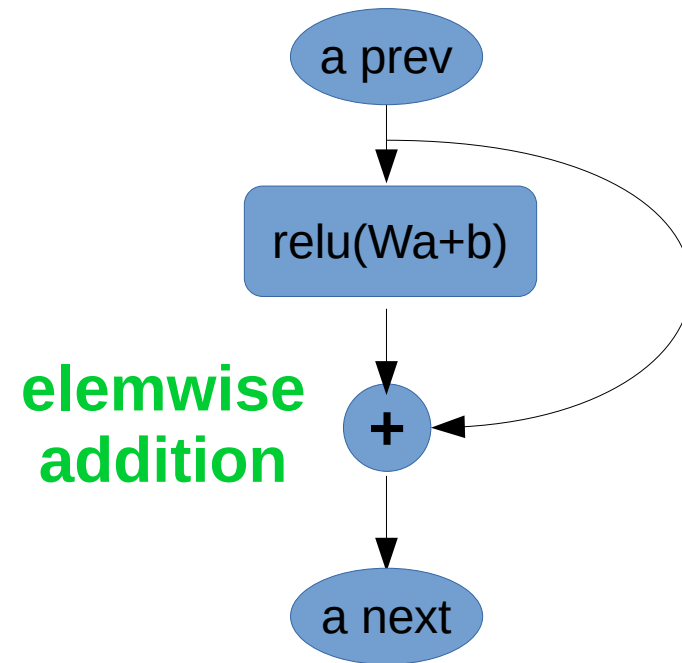
Normal layer



$$f_{w,b}(x) = \text{relu}(W \cdot a + b)$$

$$\nabla f_{w,b}(x) = \nabla \text{relu}(W \cdot a + b)$$

Residual layer



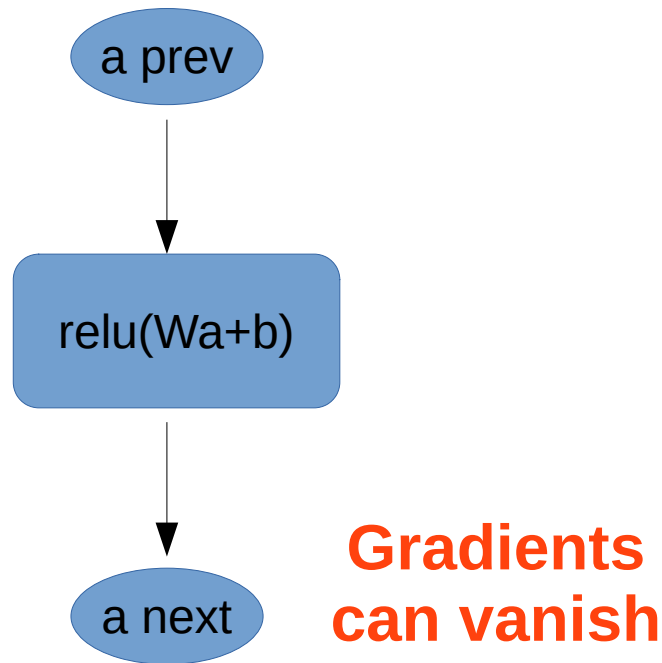
$$f_{w,b}(x) = \text{relu}(W \cdot a + b) + X$$

???

Architectures: residual network

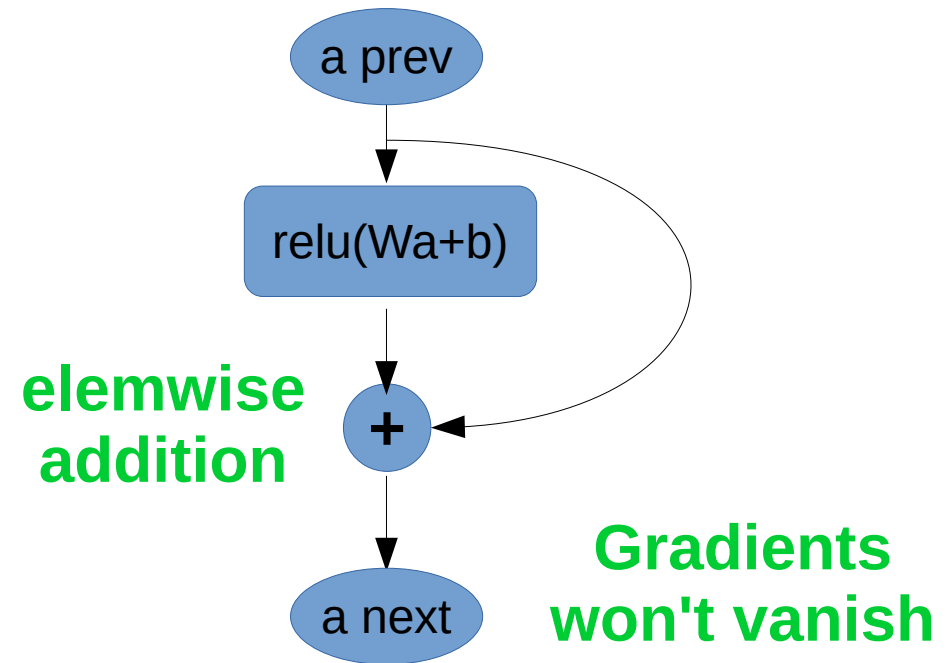
Idea: let's create a shortcut for gradients

Normal layer



$$f_{w,b}(x) = \text{relu}(W \cdot a + b)$$
$$\nabla f_{w,b}(x) = \underline{\nabla \text{relu}(W \cdot a + b)}$$

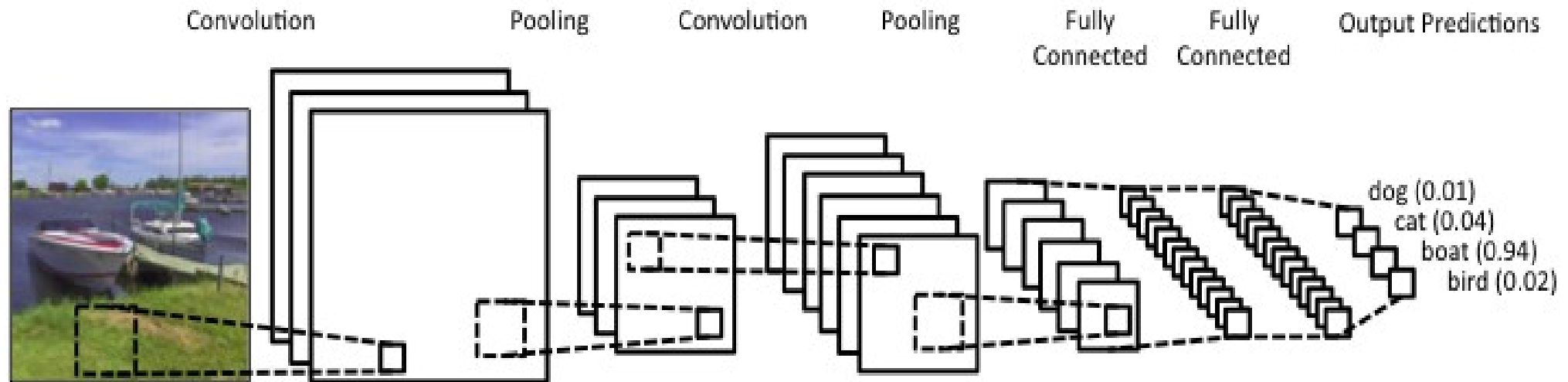
Residual layer



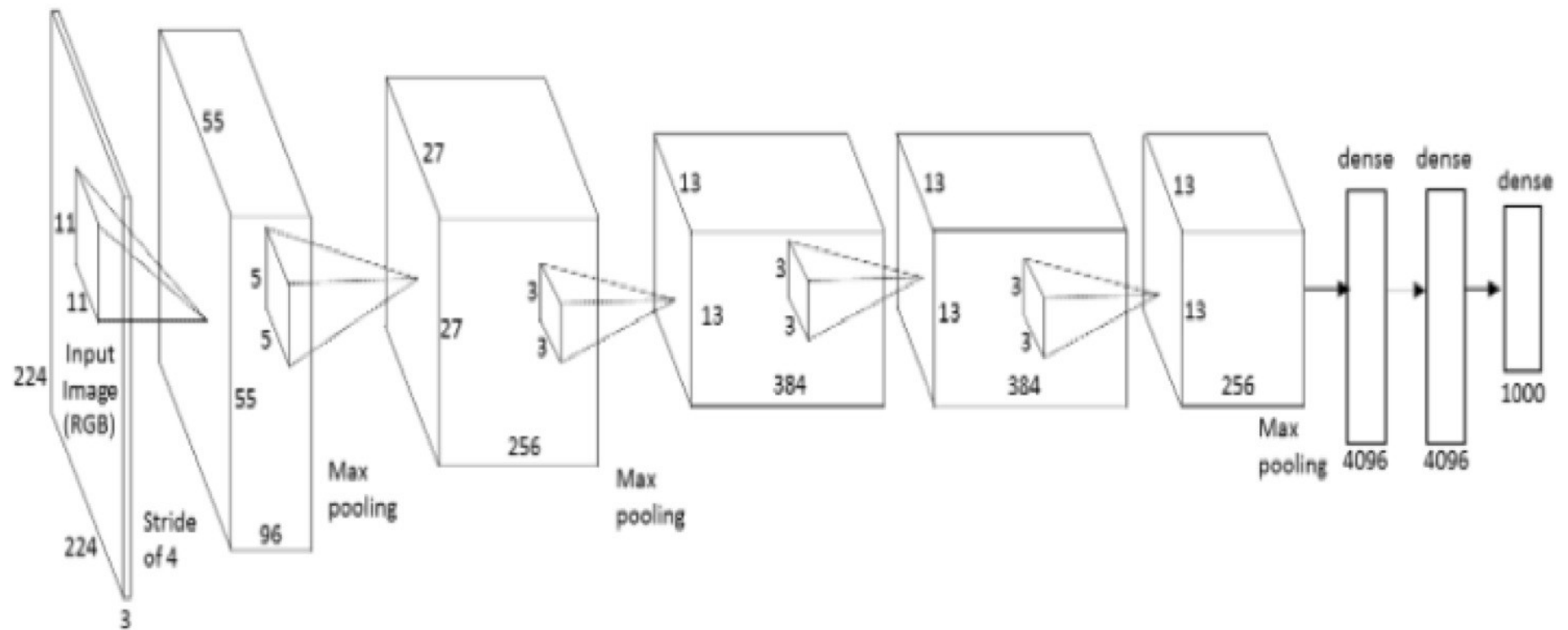
$$f_{w,b}(x) = \text{relu}(W \cdot a + b) + X$$
$$\nabla f_{w,b}(x) = \underline{\nabla \text{relu}(W \cdot a + b)} + \underline{\vec{1}}$$

BACK TO COMPUTER VISION

Previously...

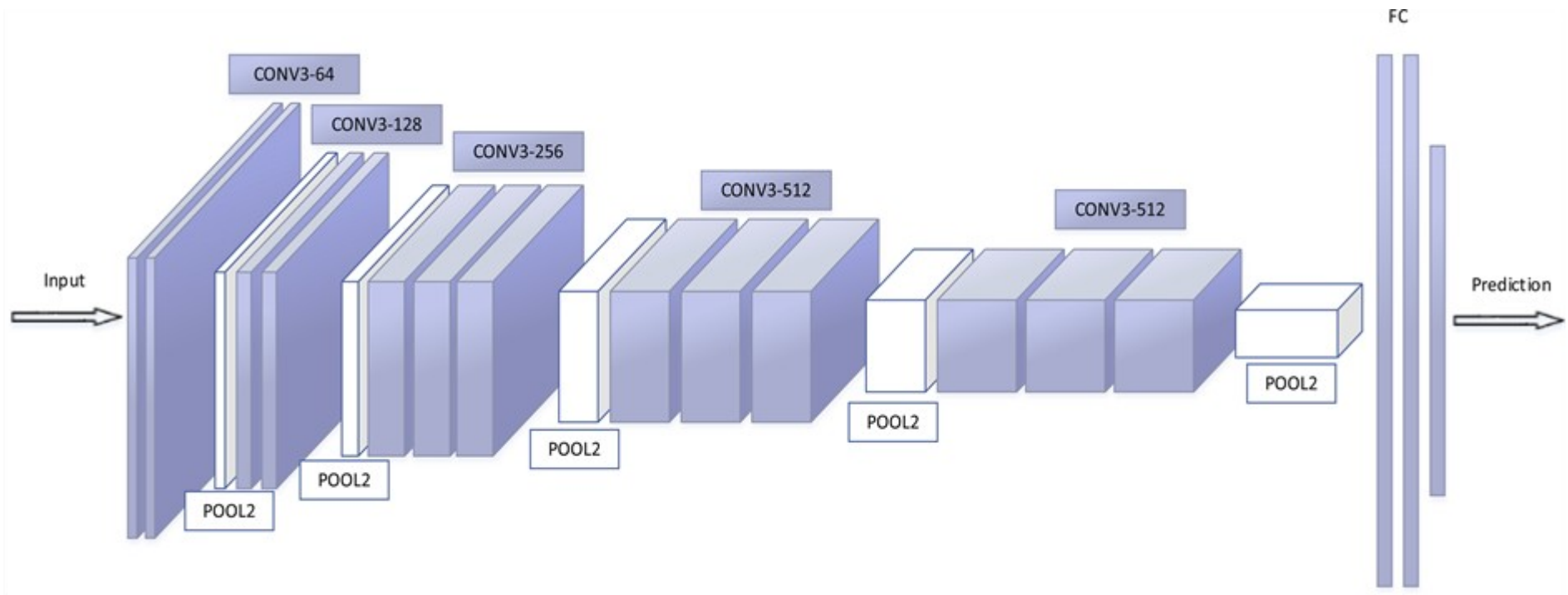


Alexnet



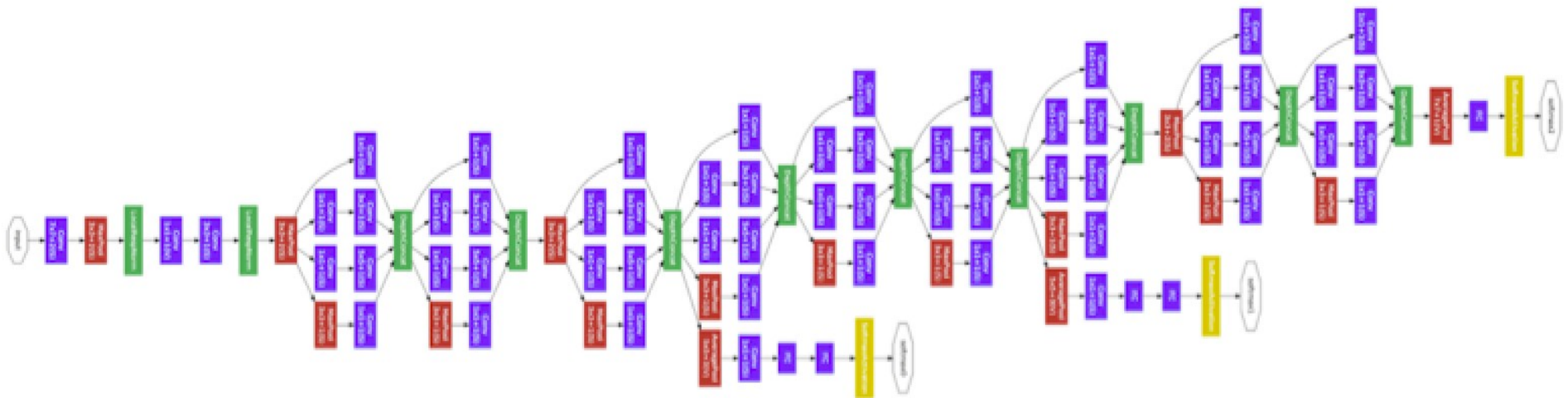
(Krizhevsky et al.)

VGG16



Karen Simonyan and Andrew Zisserman

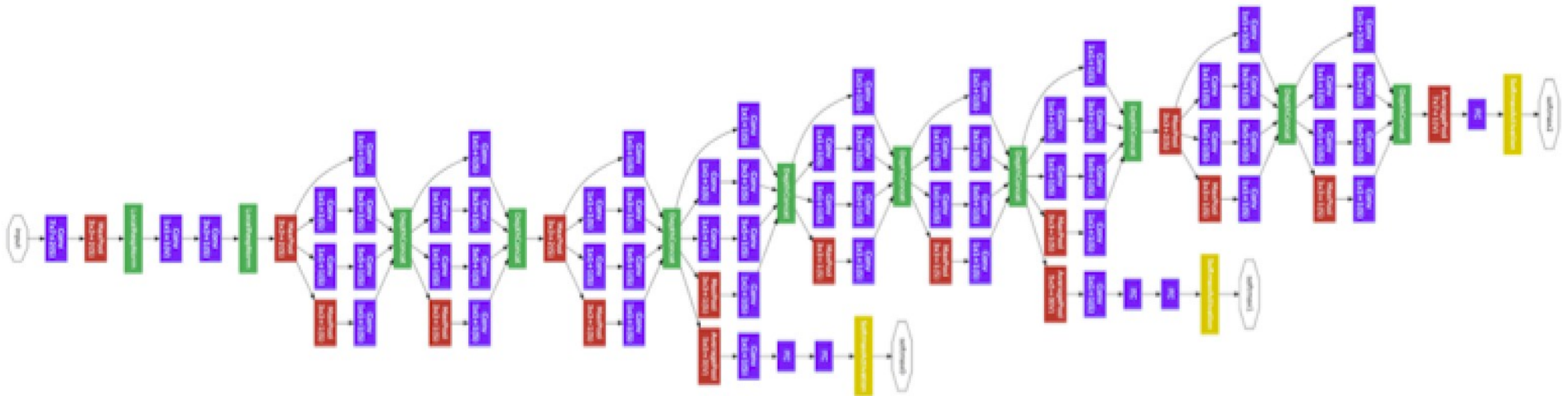
Recap: Inception



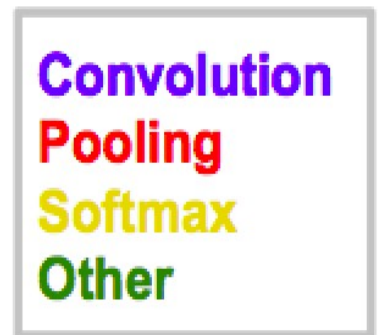
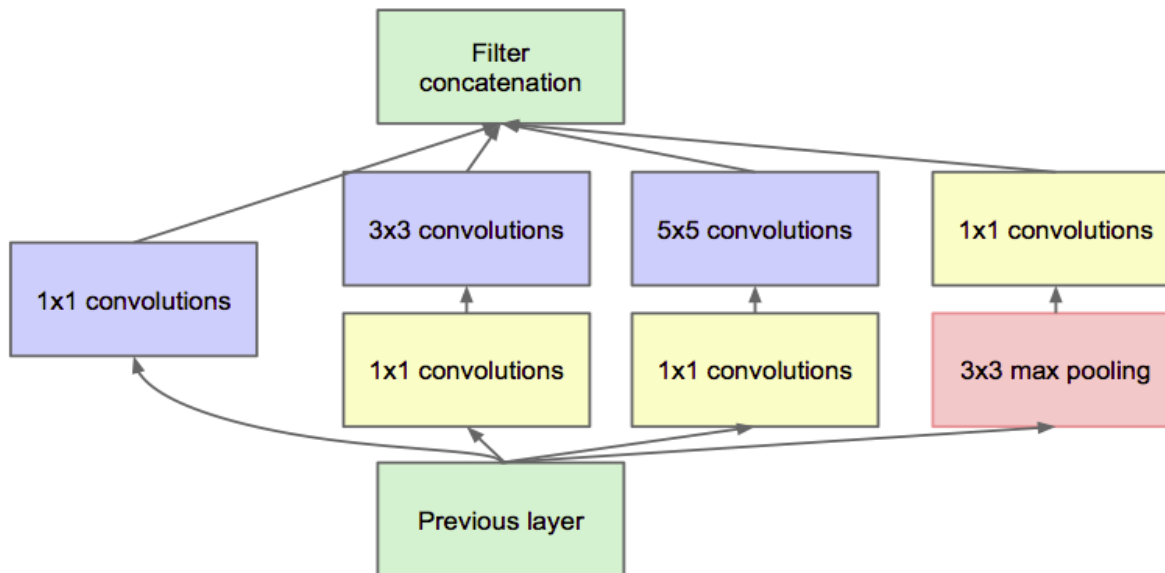
Convolution
Pooling
Softmax
Other

It is not a moon. It is a space station (c)

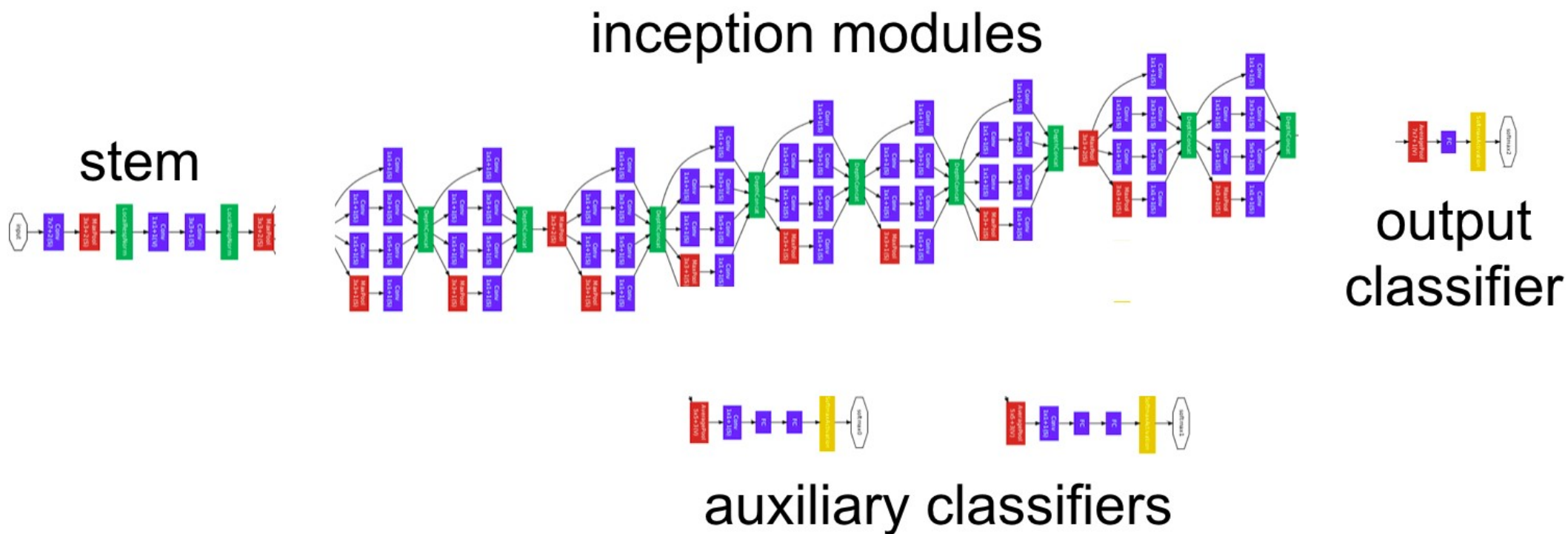
Recap: Inception



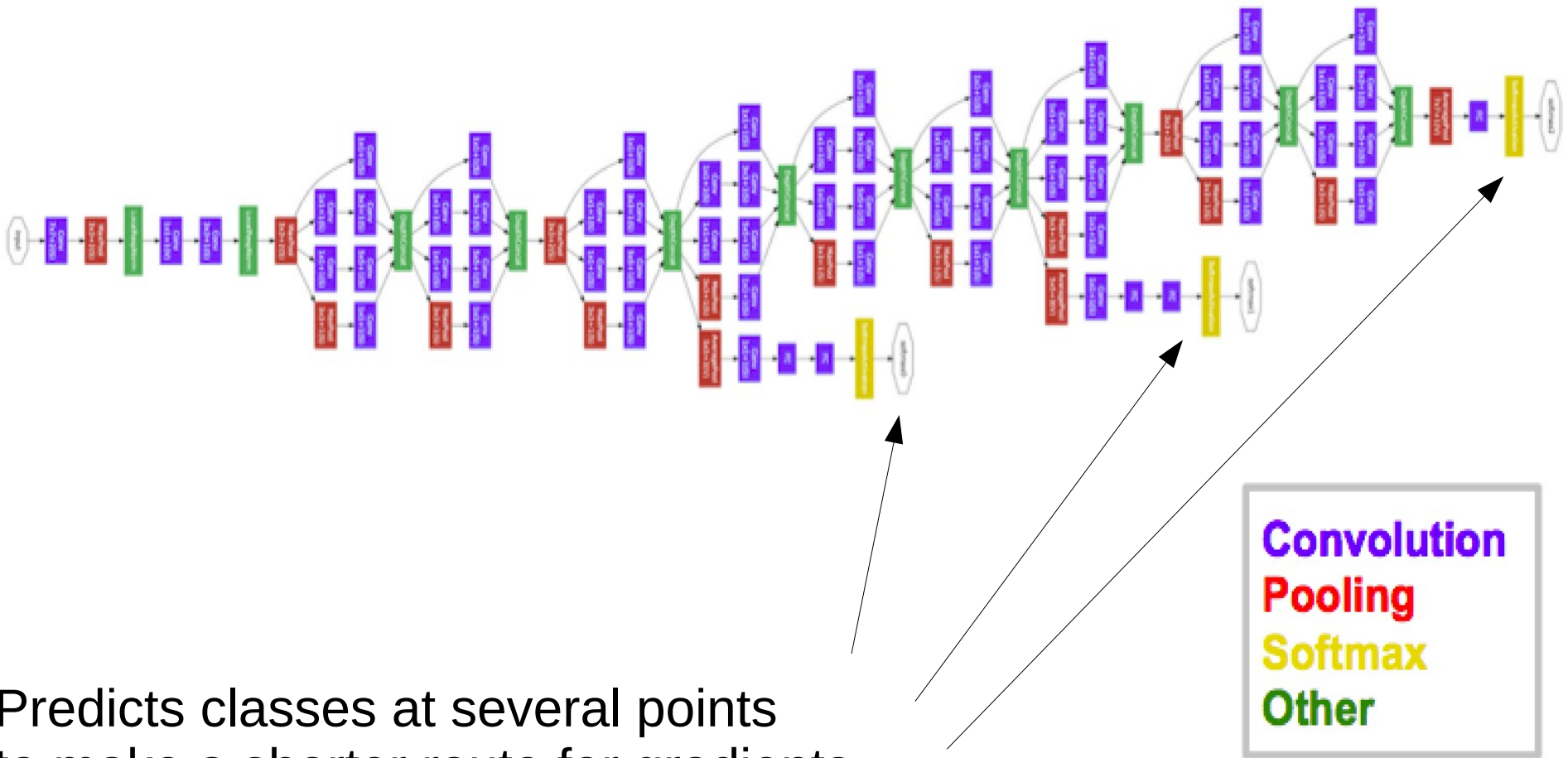
“Inception module”



Recap: Inception



Recap: Inception



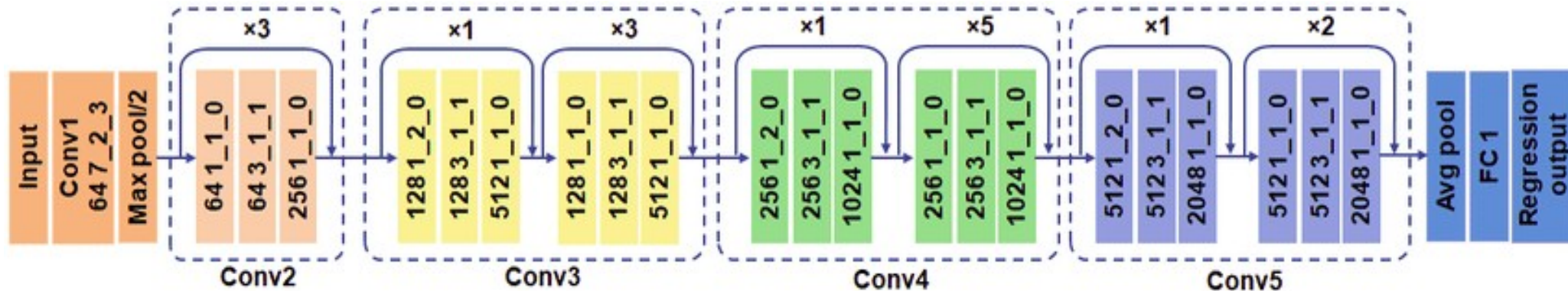
Problem with large networks

What you sign for if you stack 1000 layers:

- `MemoryError(0x...)`
- Gradients can vanish
- Gradients can explode
- Activations can vanish
- Activations can explode
- Overfits like crazy

ResNet & DenseNet

ResNet: add up activations



<https://arxiv.org/abs/1512.03385>

DenseNet: concatenate activations

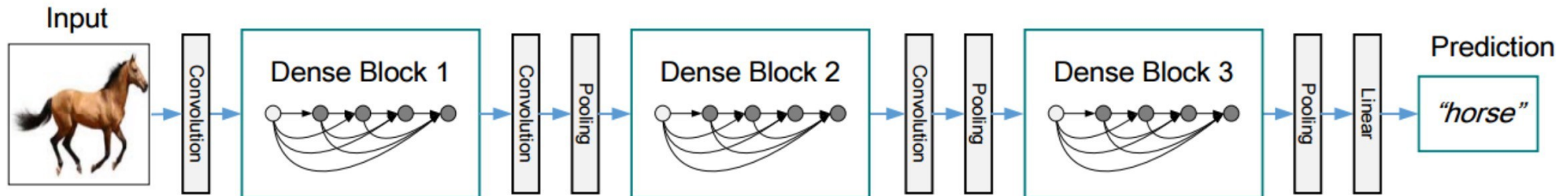


Figure 2. A deep DenseNet with three dense blocks. The layers between two adjacent blocks are referred to as transition layers and change feature map sizes via convolution and pooling.

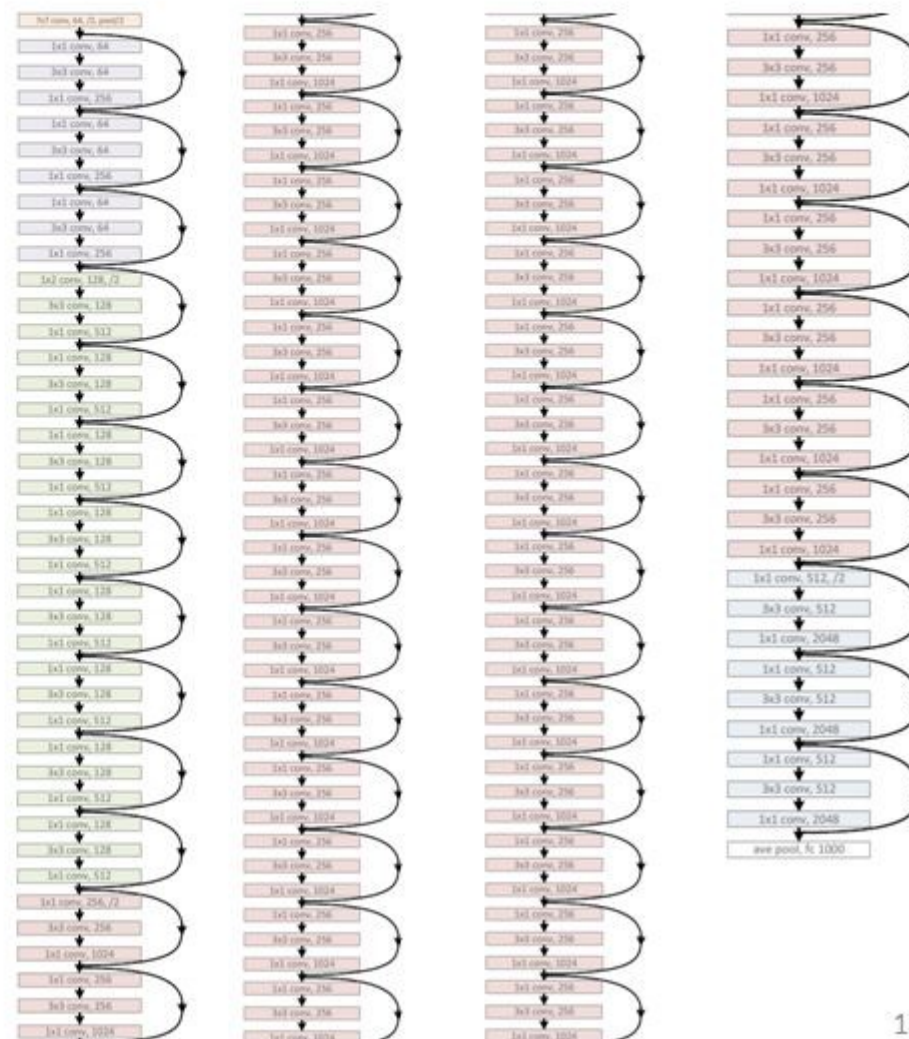
<https://arxiv.org/abs/1608.06993>

Resnet-152

Each layer learns to “modify”
previous activation

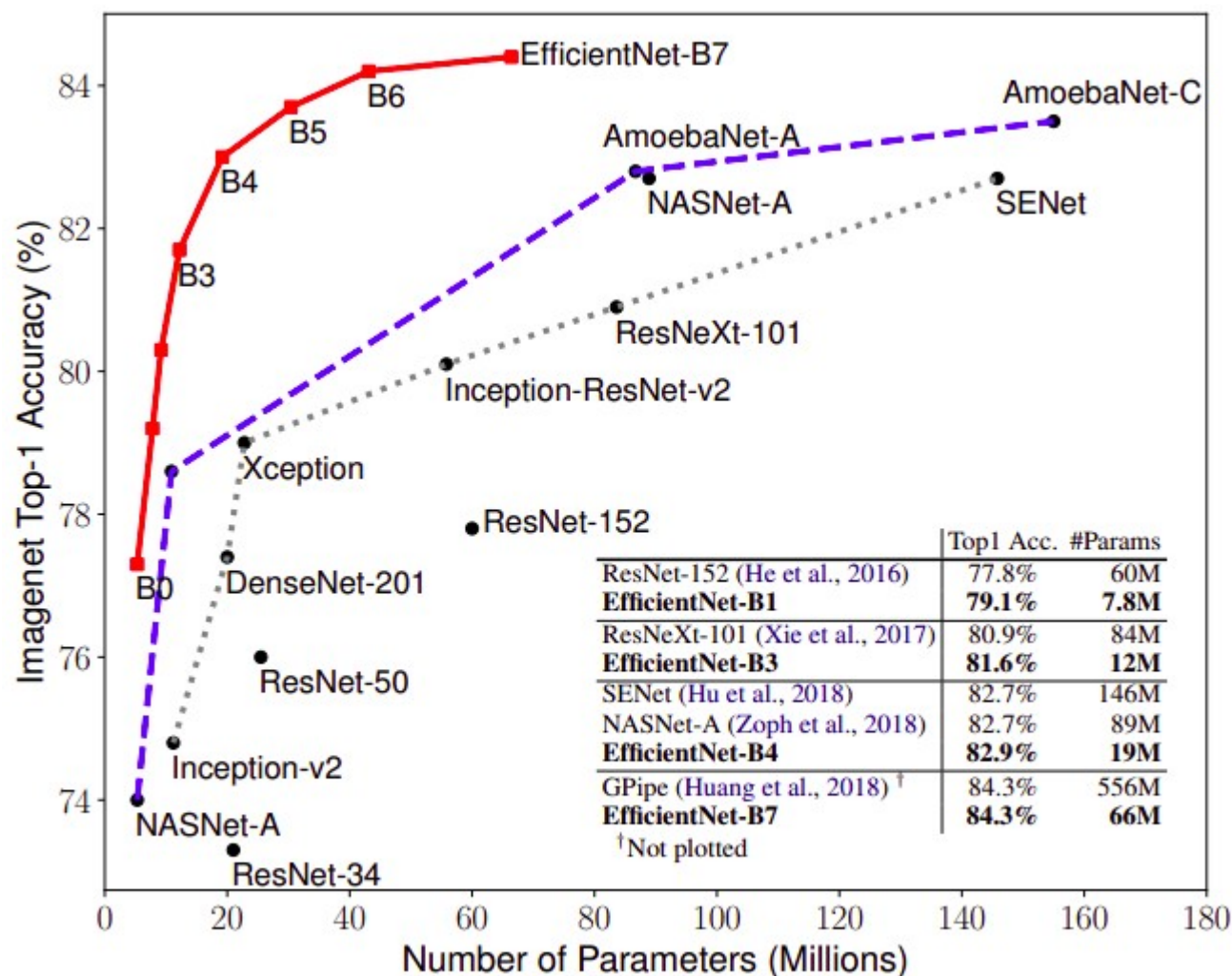
Much less parameters than
vgg16/vgg19

Can be used for other tasks:
segmentation, detection,
captioning
(just like vgg/inception/...)



Modern approach: EfficientNet

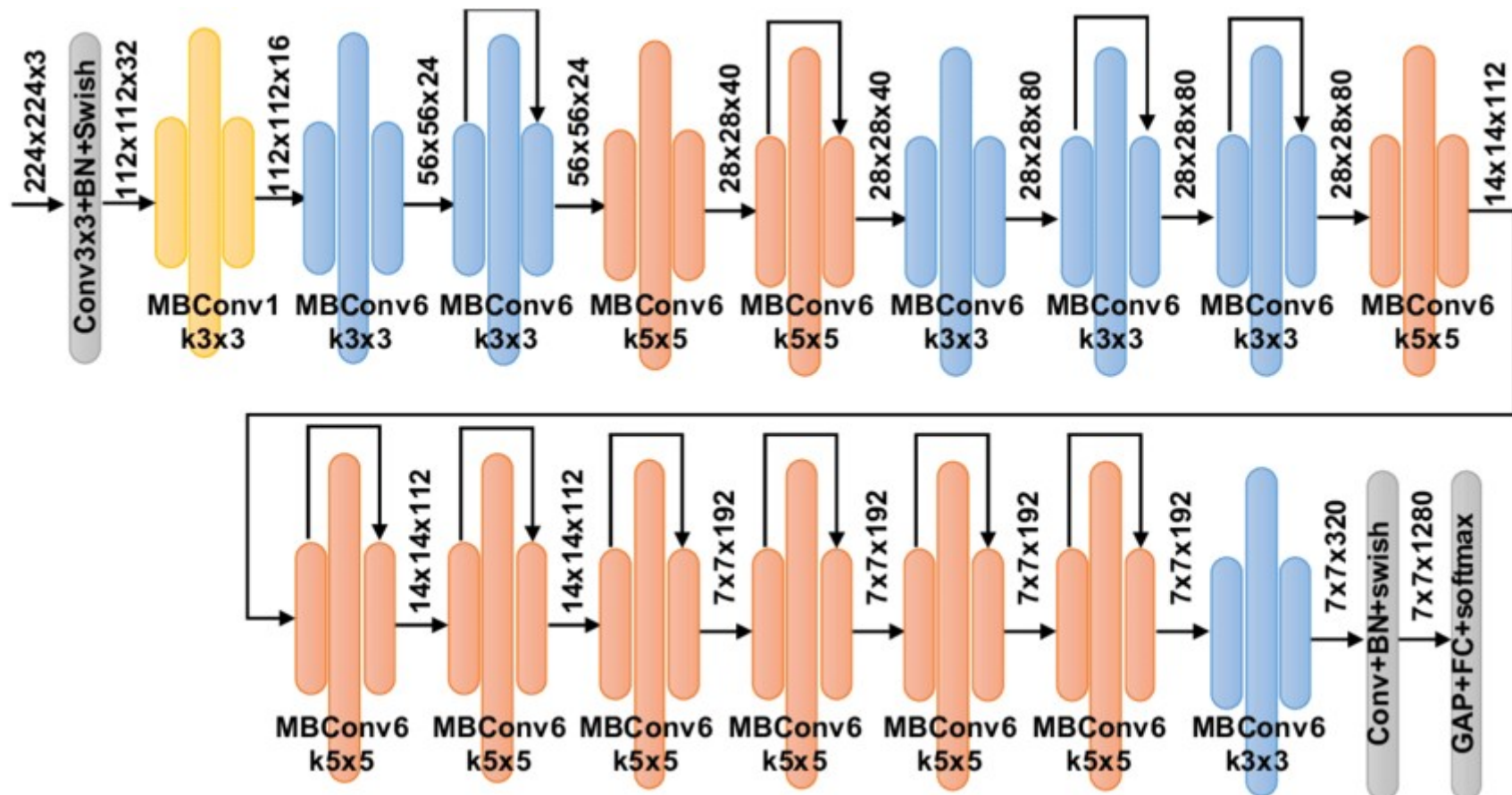
<https://arxiv.org/abs/1905.11946>



Modern approach: EfficientNet

<https://arxiv.org/abs/1905.11946>

- Designed base network with neural architecture search (not the first paper to do so)



Modern approach: EfficientNet

<https://arxiv.org/abs/1905.11946>

- Found optimal scaling ratio for depth/width/res

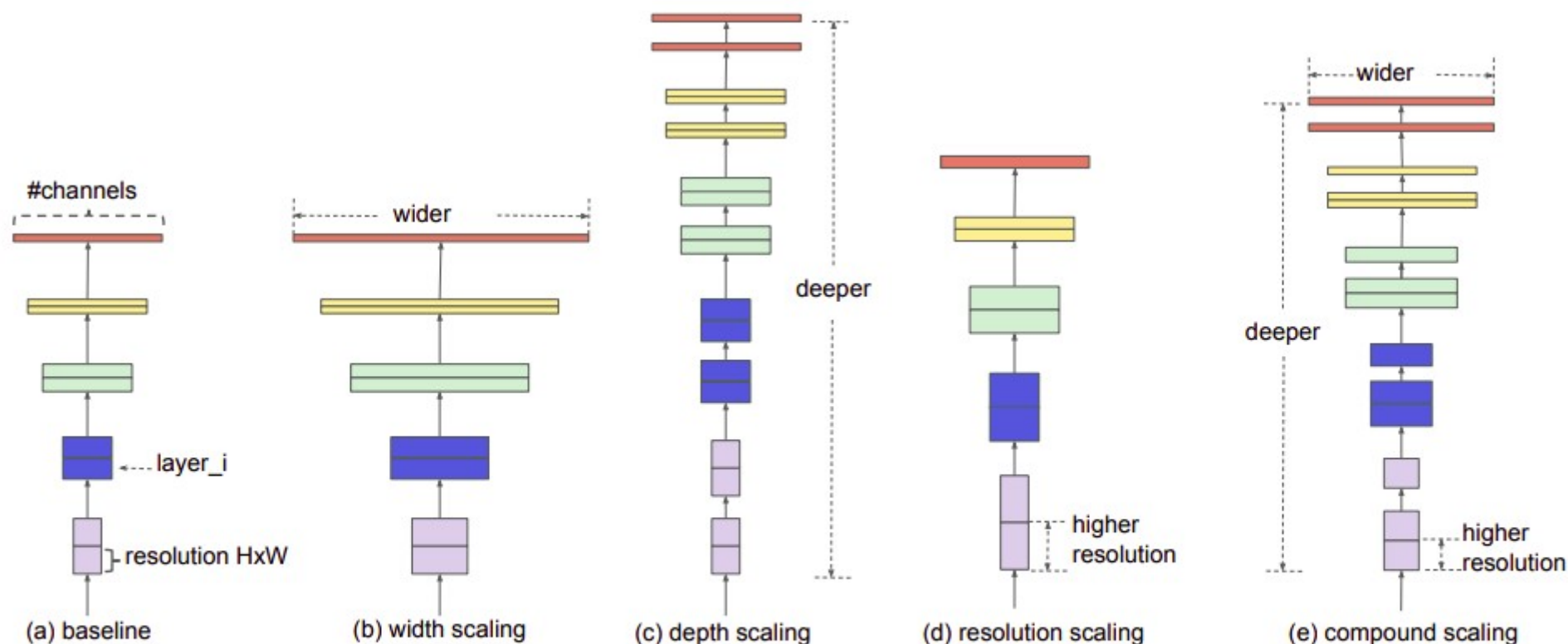
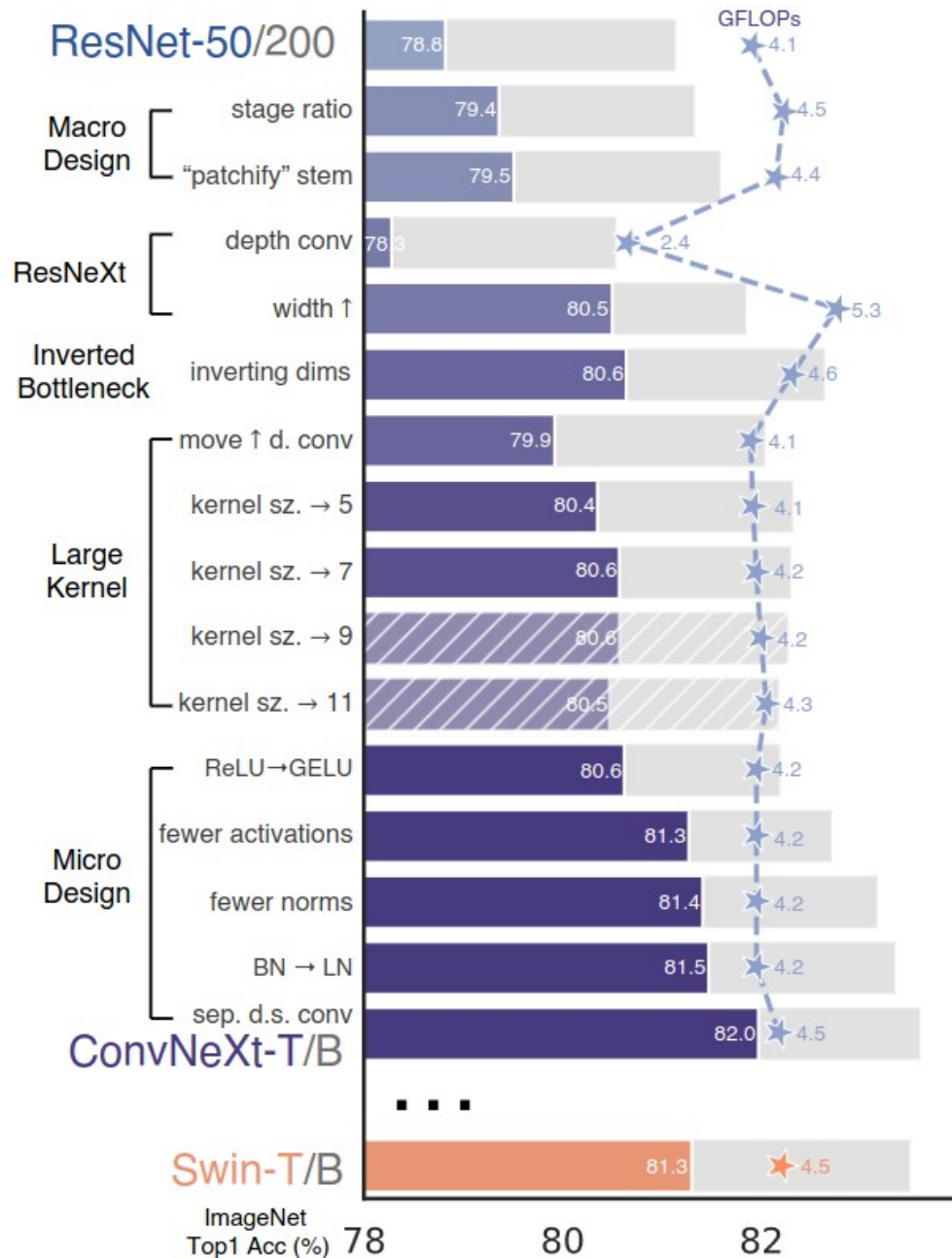


Figure 2. Model Scaling. (a) is a baseline network example; (b)-(d) are conventional scaling that only increases one dimension of network width, depth, or resolution. (e) is our proposed compound scaling method that uniformly scales all three dimensions with a fixed ratio.

Modern approach: ConvNext

<https://arxiv.org/abs/2201.03545>



1. gather architecture insights and training tricks from post-resnet papers
2. apply them all to resnet and analyze which work best
3. ???
4. SOTA!!!

Modern stuff

- Architectures:

EfficientNetV2 <https://arxiv.org/abs/2104.00298>

CNN for 2020 <https://arxiv.org/abs/2201.03545>

Vision transformers - later

- Interpretability

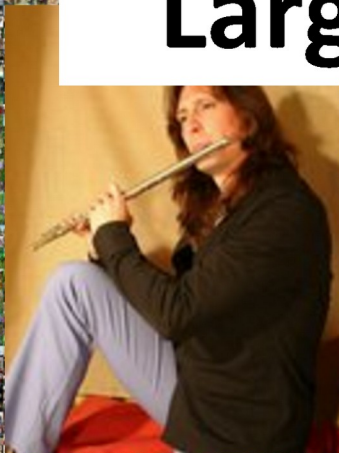
<https://distill.pub/2018/building-blocks>

Note: data augmentation



- Idea: we can get N times more data by tweaking images.
- If you rotate cat image by 15° , it's still a cat
- Rotate, crop, zoom, flip horizontally, add noise, etc.
- Sound data: add background noises

Large-scale recognition



CIFAR



- 60k images
- 10 classes (left)
- 32x32 RGB
- subclasses

aquatic mammals

fish

flowers

food containers

beaver, dolphin, otter, seal, whale

aquarium fish, flatfish, ray, shark, trout

orchids, poppies, roses, sunflowers, tulips

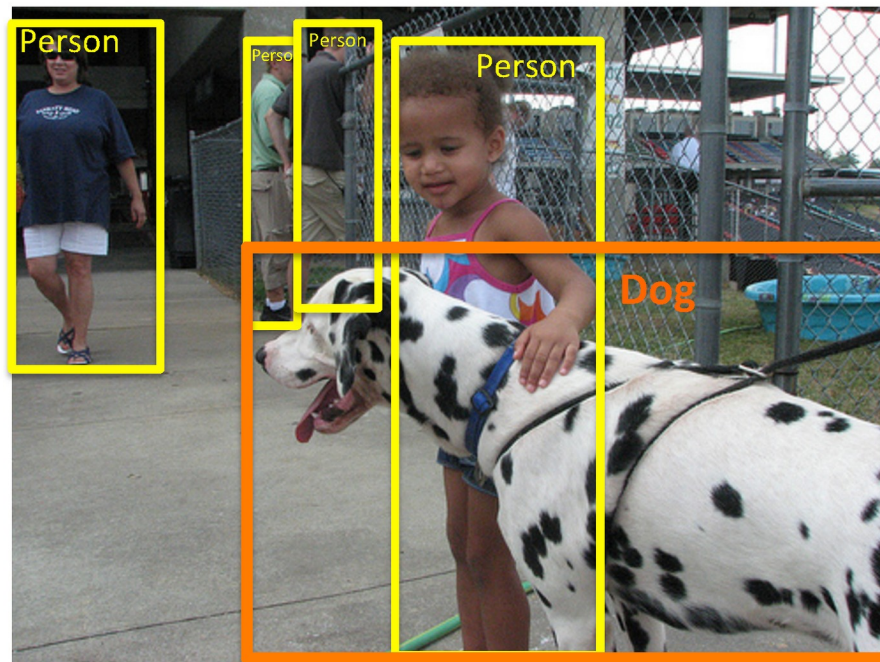
bottles, bowls, cans, cups, plates

IMAGENET Large Scale Visual Recognition Challenge (ILSVRC) 2010-2014

200 object classes
1000 object classes

456,567 images
1,431,167 images

DET
CLS-LOC



<http://image-net.org/challenges/LSVRC/>

Variety of object classes in ILSVRC

DET

CLS-LOC

birds



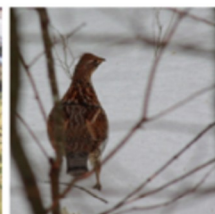
bird



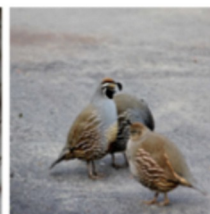
flamingo



cock



ruffed grouse



quail



partridge . . .

bottles



bottle



pill bottle



beer bottle



wine bottle



water bottle



pop bottle . . .

cars



car



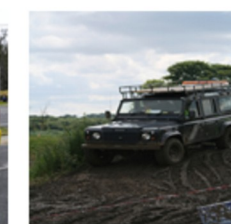
race car



wagon



minivan



jeep



cab . . .

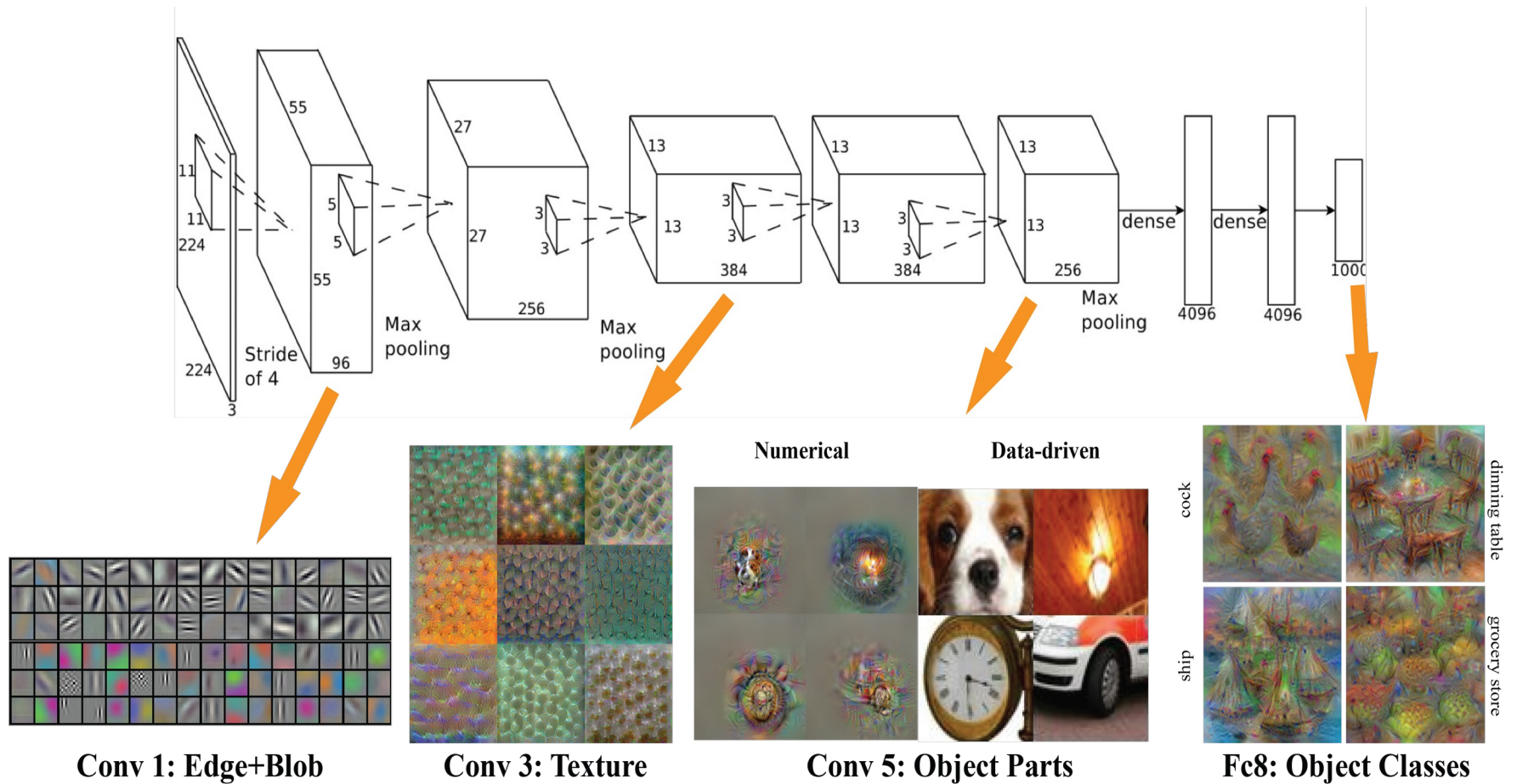
The only problem: you only have 5k images
e.g. transport vs military aircrafts

Ideas?

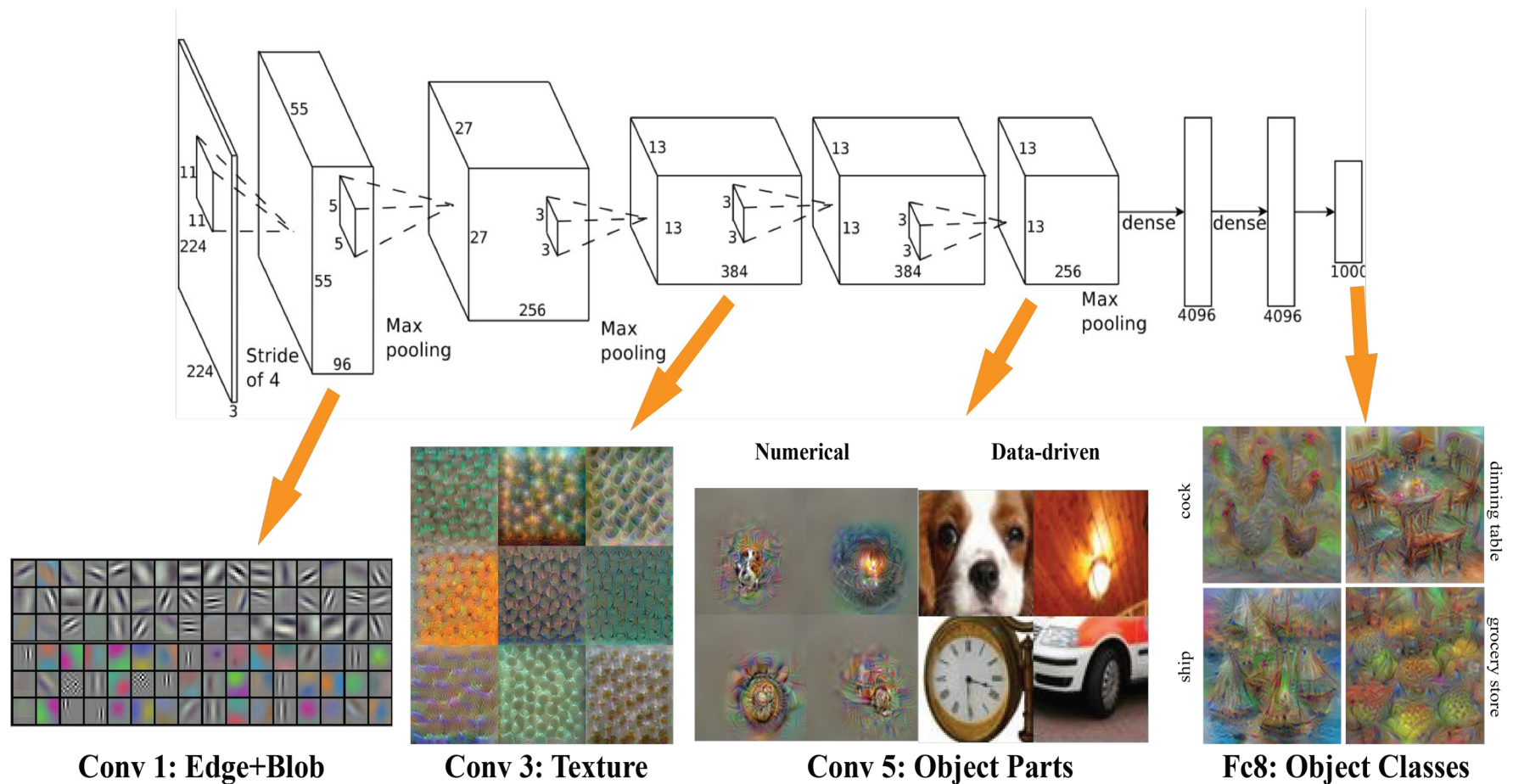
Traditional solutions

- Regularize really hard
- Super small network
- Data augmentation
- Whatever

Feature learning

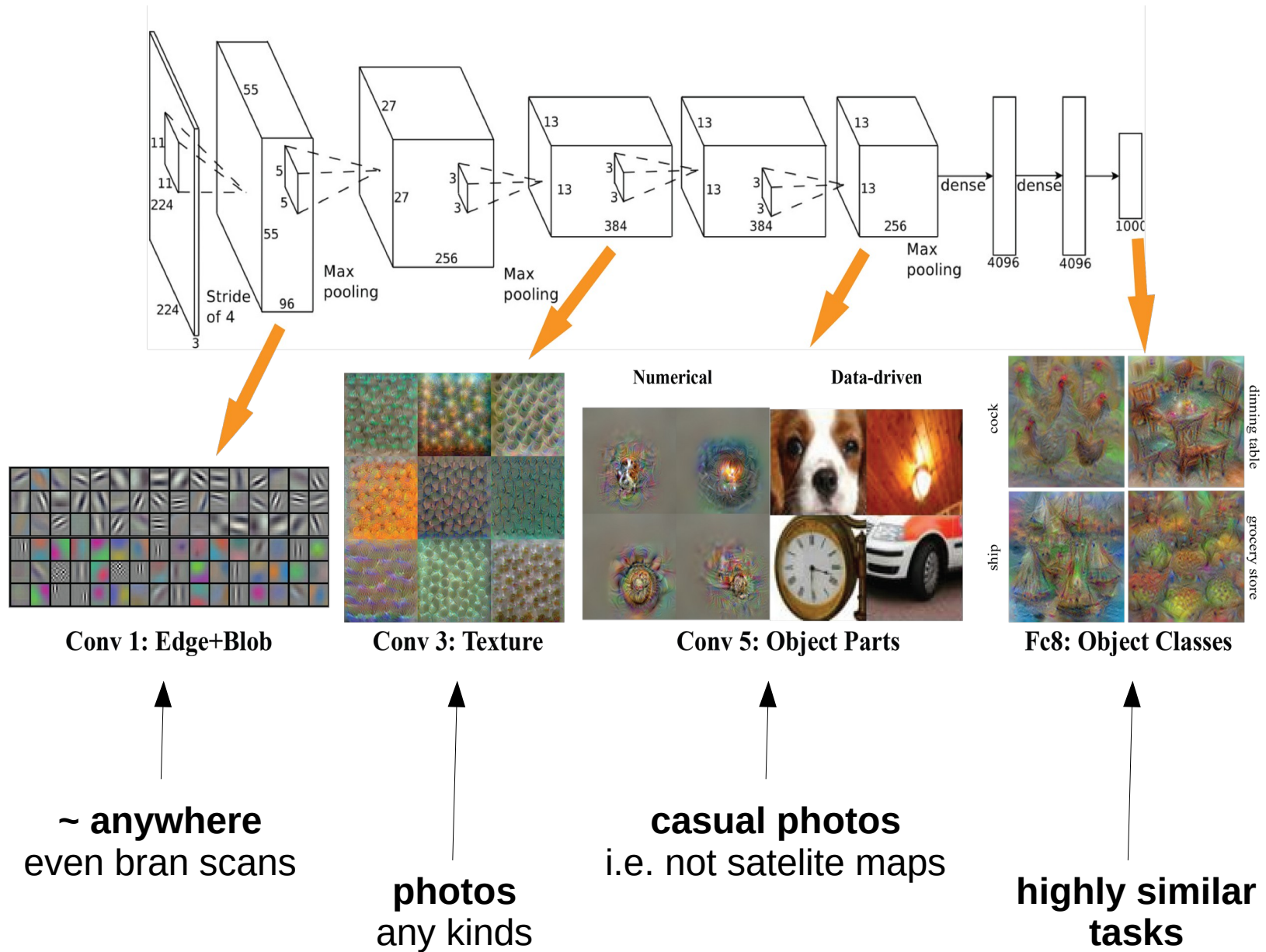


Feature learning



Idea: let's pre-train network on a larger dataset

Can we use them?



Pre-training

- 1. Train a network on large dataset

cifar X

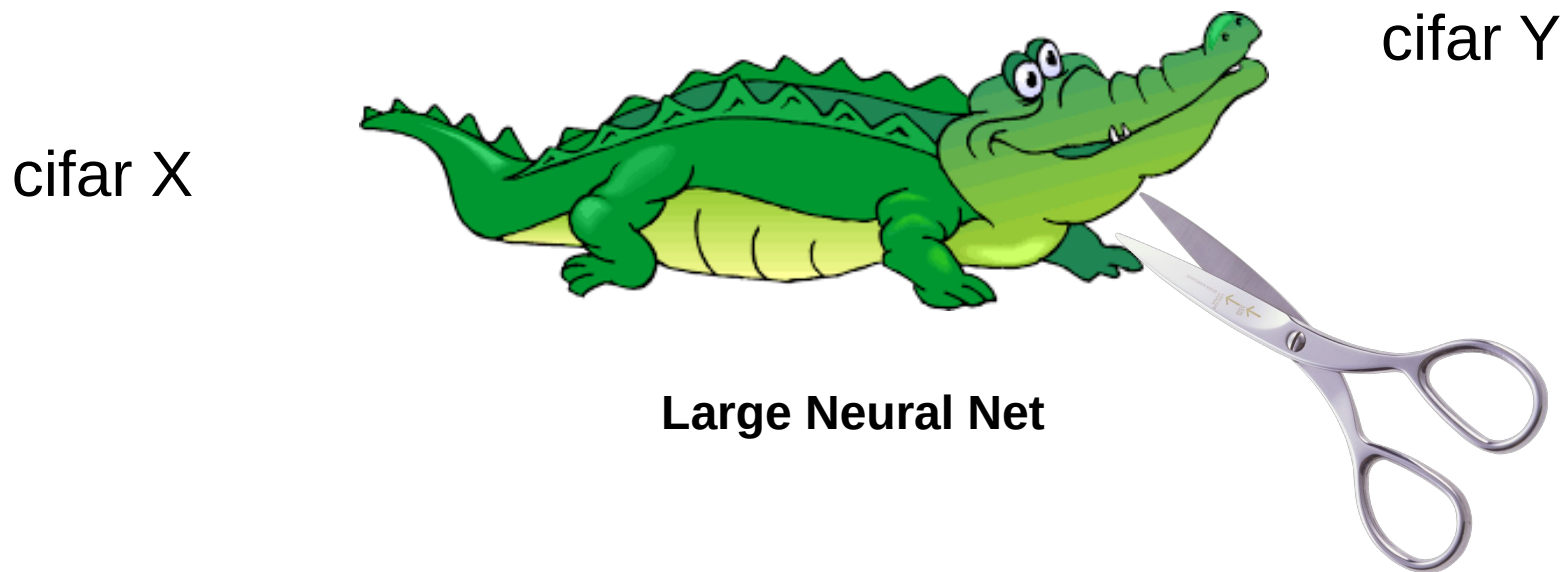


cifar Y

Large Neural Net

Pre-training

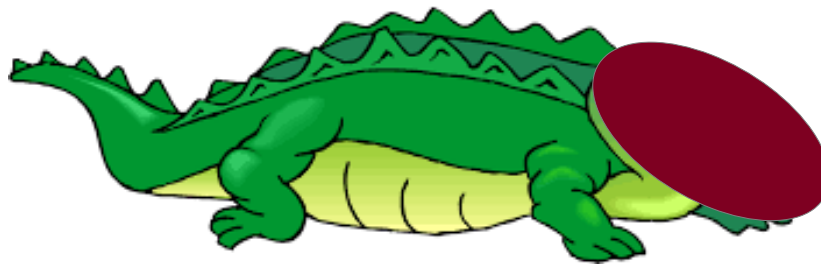
- 1. Train a network on large dataset
- 2. Take some intermediate layer



Pre-training

- 1. Train a network on large dataset
- 2. Take some intermediate layer

cifar X

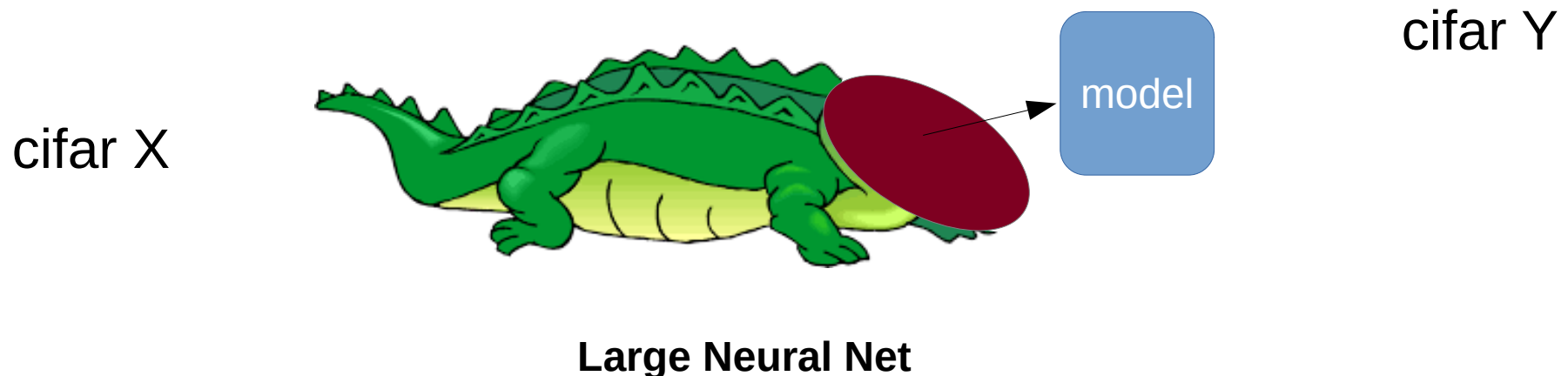


Large Neural Net

cifar Y

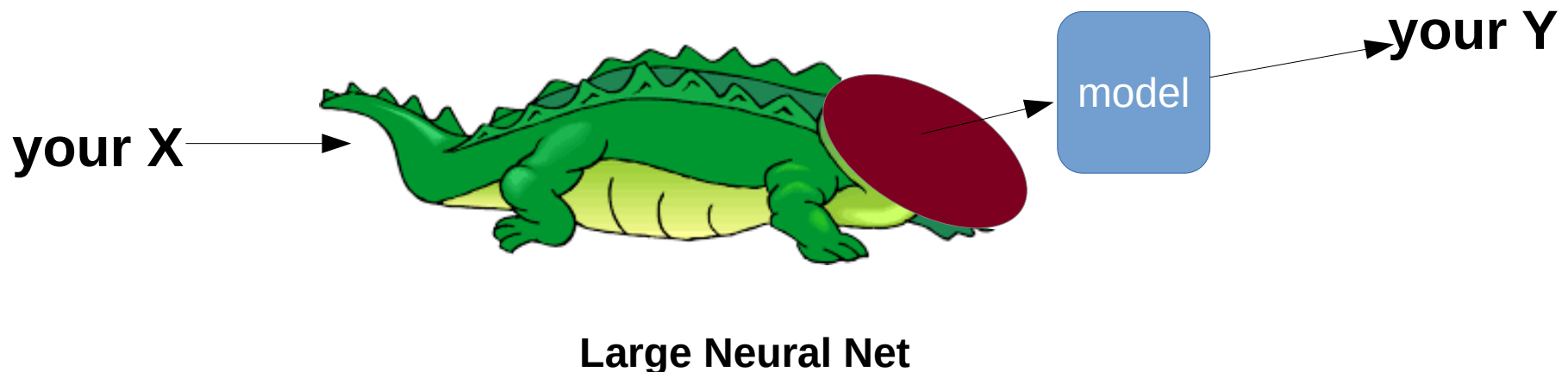
Pre-training

- 1. Train a network on large dataset
- 2. Take some intermediate layer
- 3. Build model on top of it



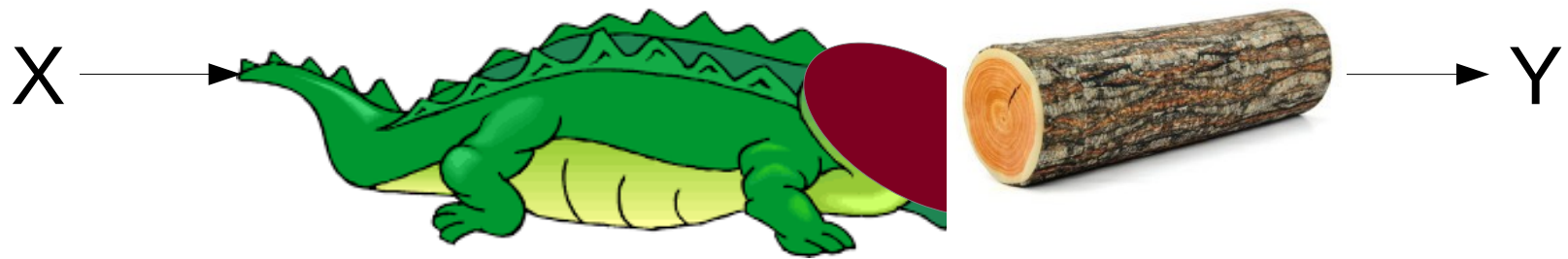
Pre-training

- 1. Train a network on large dataset
- 2. Take some intermediate layer
- 3. Build model on top of it
- 4. Train model for your objective



Reusing features

- Chop off “head”
- use “neck” as feature extractor
- Train ANY classifier
 - even random forest will do



Large Neural Net

Fine-tuning

- Chop off “head”
- “freeze” body (consider constant)
- Build new neural network in it's place
- Train “head” only for several iterations
- Un-freeze body and train full network



Large Neural Net

**Q: What if we only have
1-5 images for each class?**

Few-shot learning with KNN

Idea: run KNN over frozen pre-trained model

● ● ● - training images, color = class

● auto

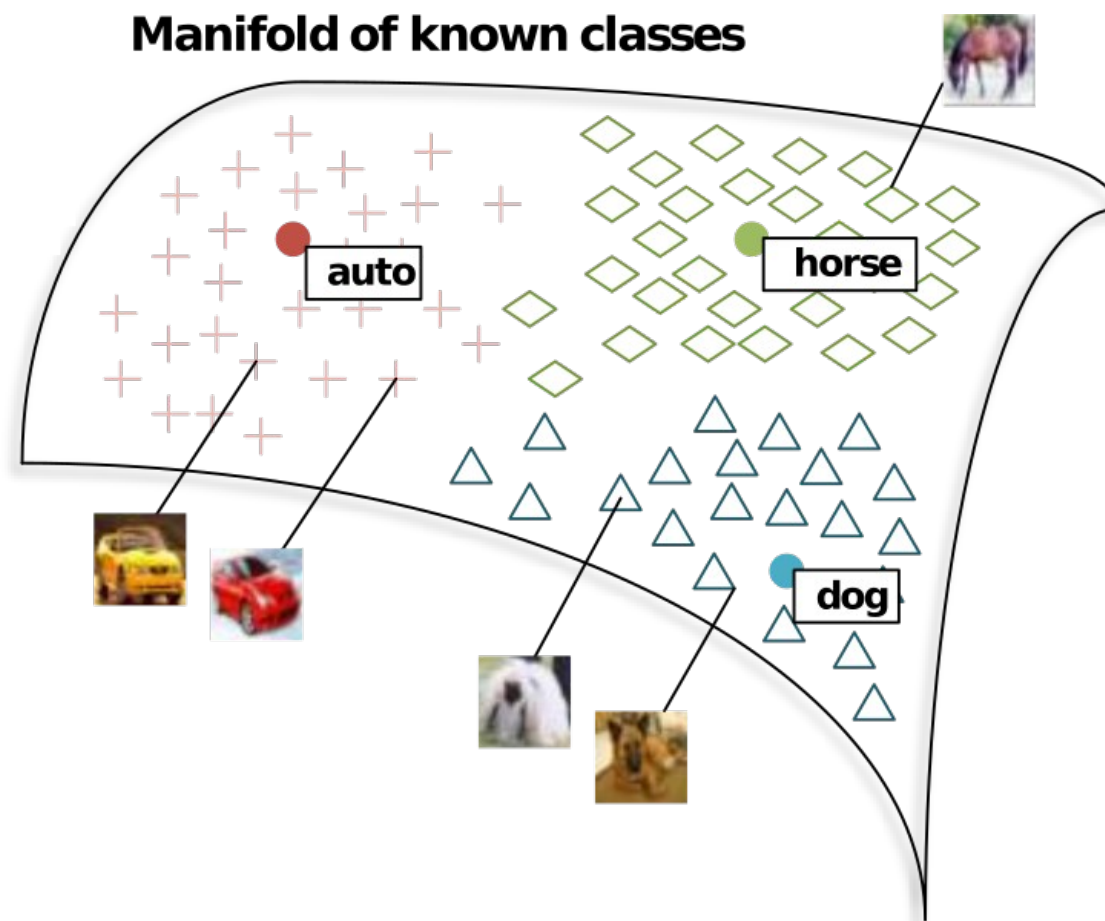
● horse

● dog

Few-shot learning with KNN

Idea: run KNN over frozen pre-trained model

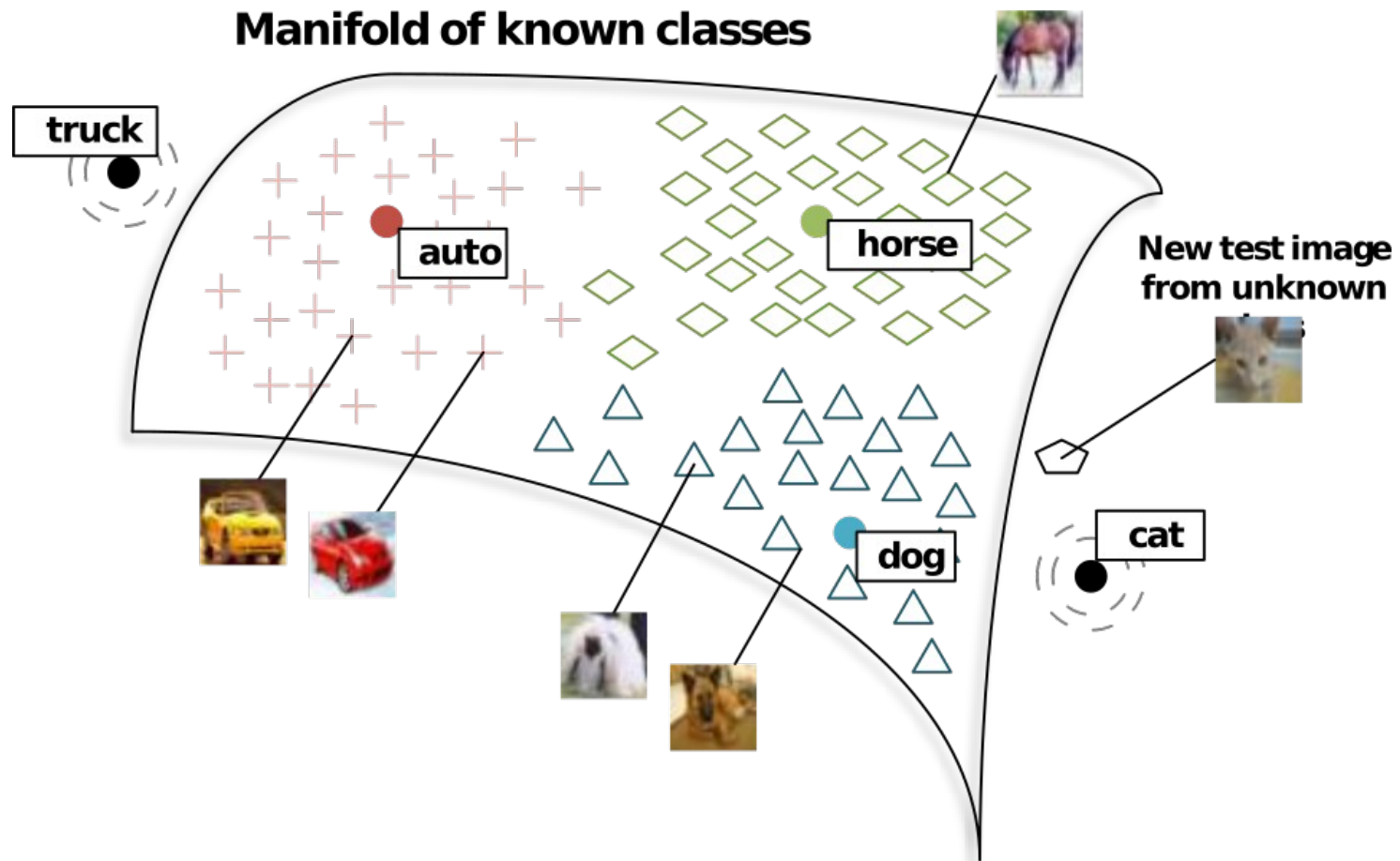
● ● ● - training images, color = class



Few-shot learning with KNN

Idea: run KNN over frozen pre-trained model

● ● ● - training images, color = class



**What if we do not have
any labeled dataset?**

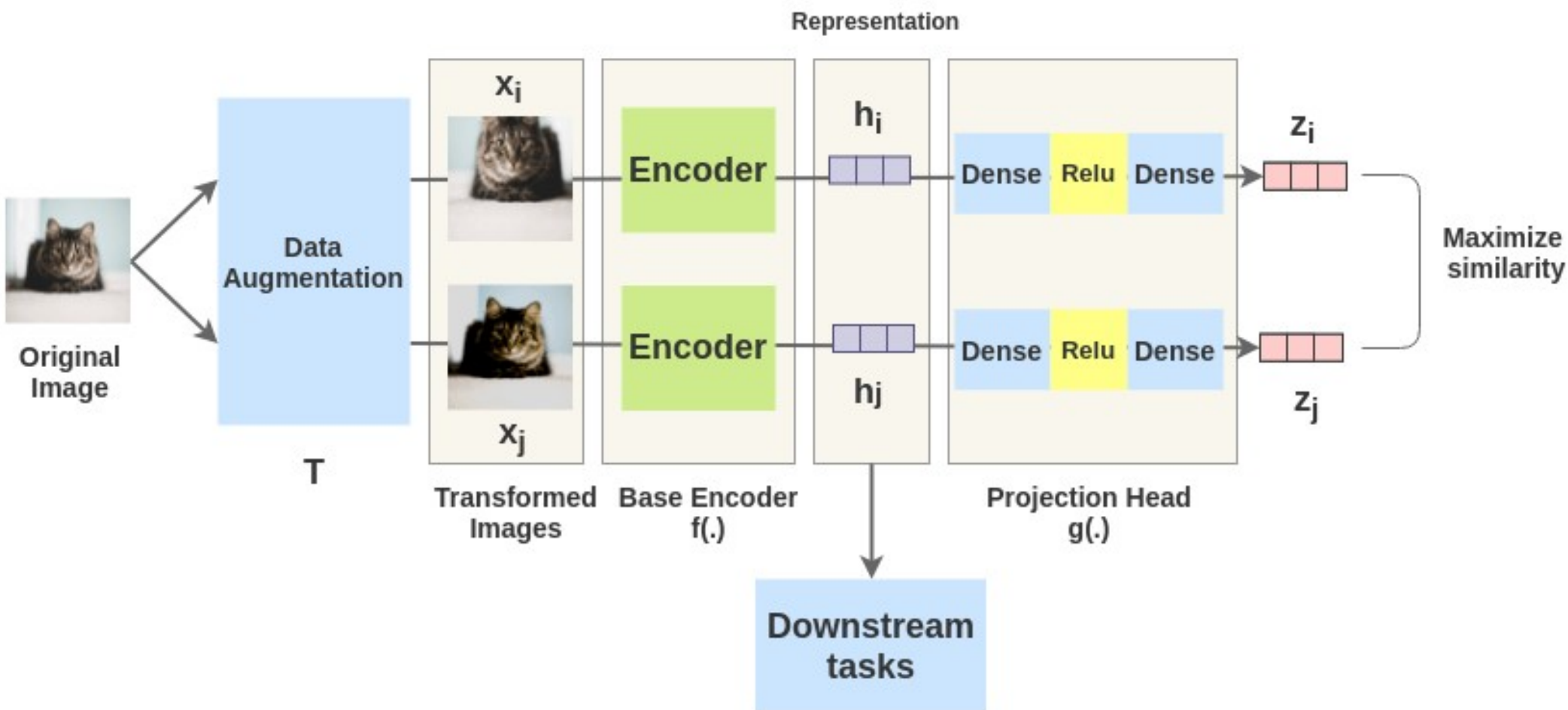
**What if we do not have
any ~~labeled~~ dataset?**

Can we learn features from just images?

Self-supervised learning

- SimCLR: <https://arxiv.org/abs/2002.05709>

SimCLR Framework

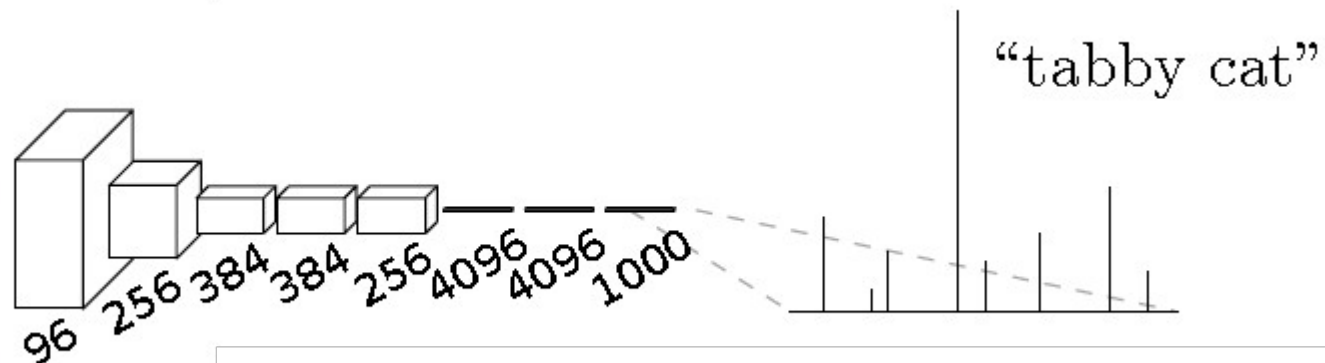


Alright, we did it!

What else can we do besides image
classification?

Semantic segmentation

- Classification

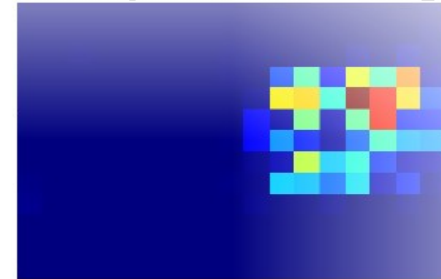


- Segmentation/Detection



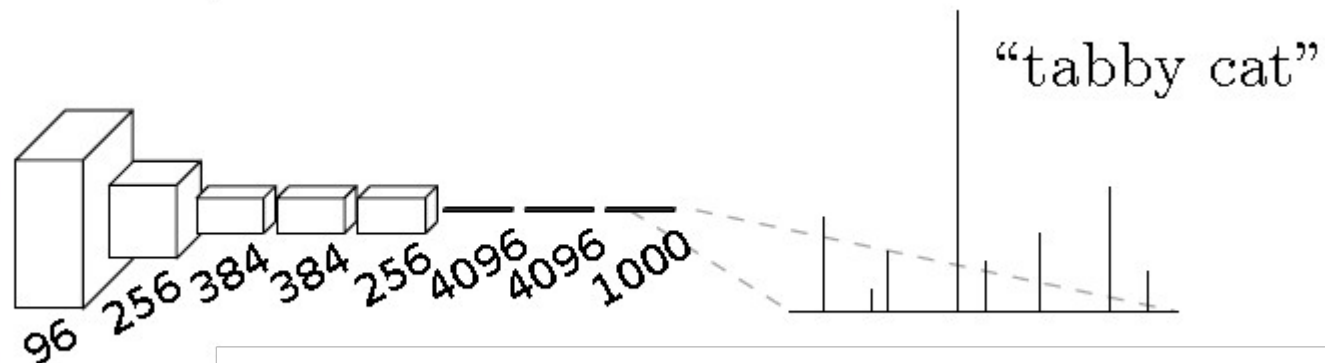
???
Ideas on how to
predict that?

tabby cat heatmap



Semantic segmentation

- Classification



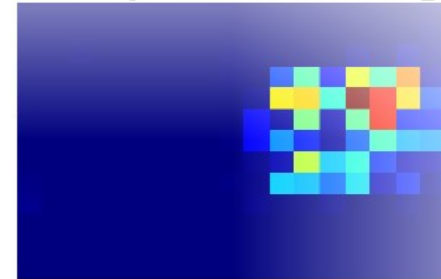
- Segmentation/Detection (naïve)



Apply to each spot

→
"convolution with whole
NN as a filter"

tabby cat heatmap



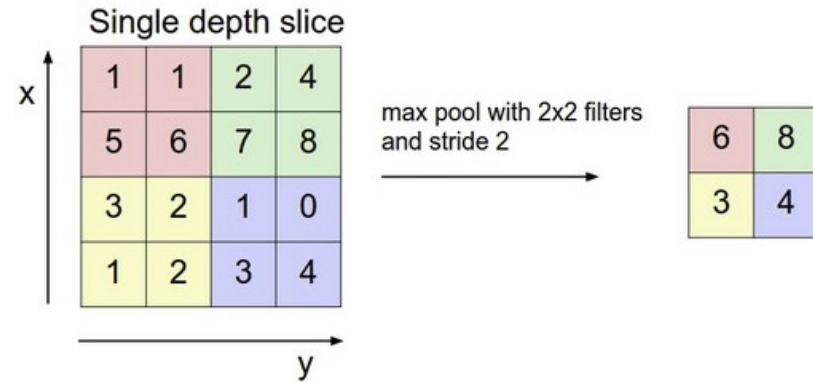
What if output must have same resolution as in the input image? Or even bigger?

How can we **increase** activation size?

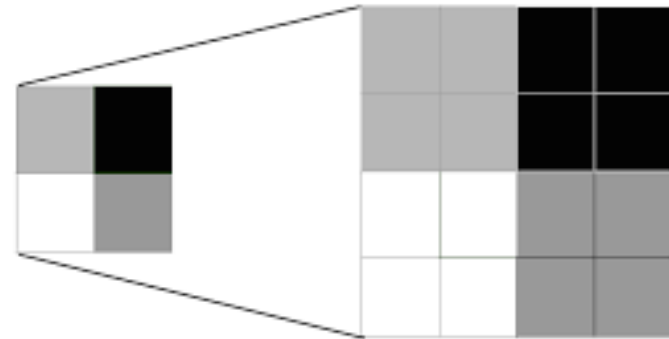
Ideas?

Upscale

- Pooling =
 - take $(n*m)$ region,
 - output one / channel

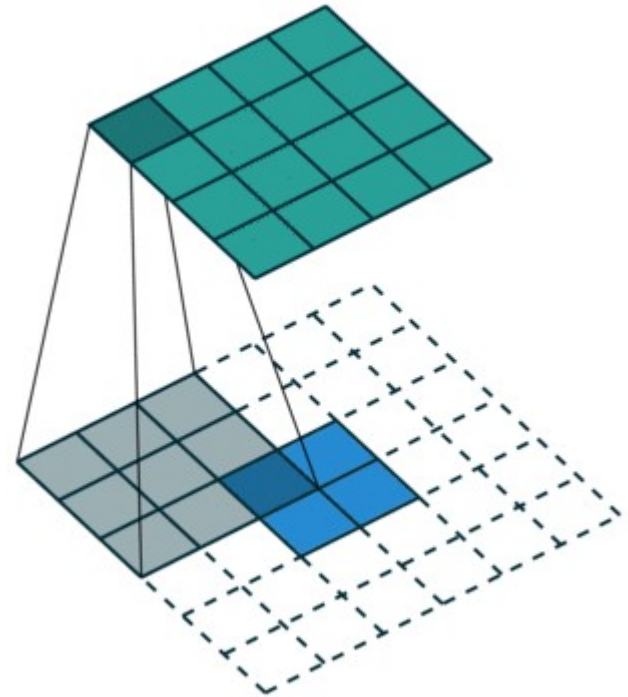


- Upscaling / Unpooling / Upsampling
 - Take one value
 - Output $(n*m)$ region
 - Several strategies



Deconvolution

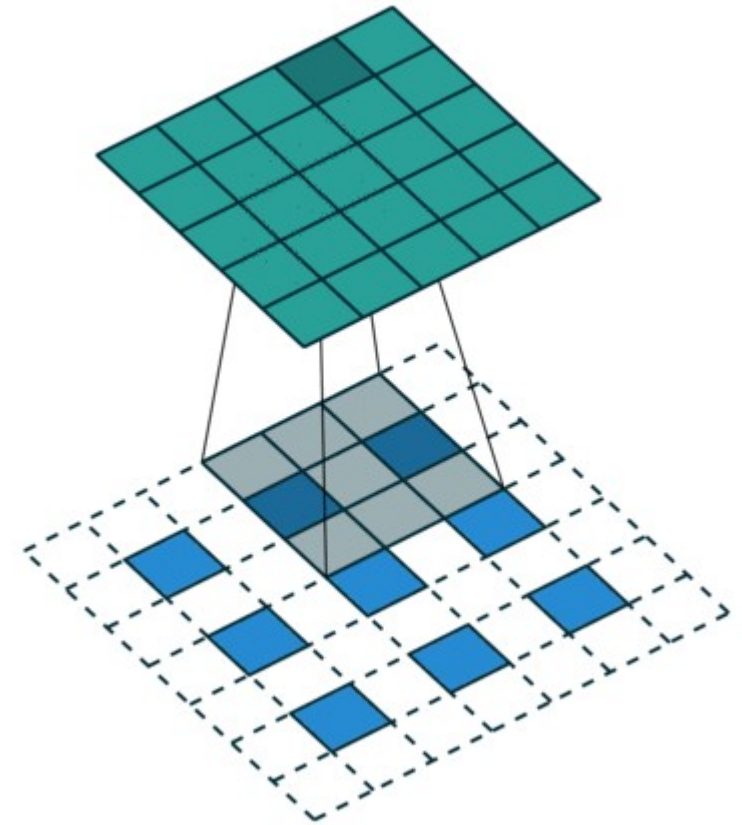
- The “inverse” of convolution
 - = transposed convolution
 - ~ wide convolution
- `deconv(conv(img))`
- preserves image shape
- Stride ~ upscale



deconvolution

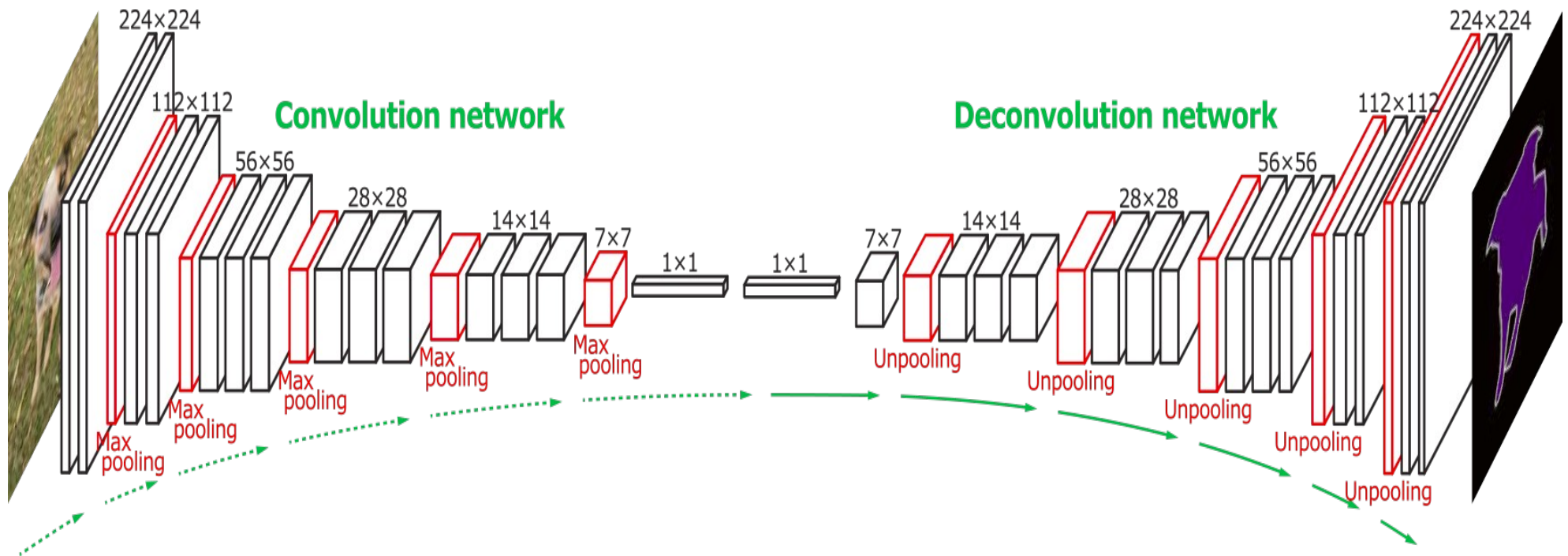
Deconvolution

- The “inverse” of convolution
 - = transposed convolution
 - ~ wide convolution
- `deconv(conv(img))`
- preserves image shape
- Stride ~ upscale



strided deconvolution

Fully-convolutional



The U-net

- Add connections between layers of similar abstraction
- The idea is similar to gaussian/laplacian pyramids

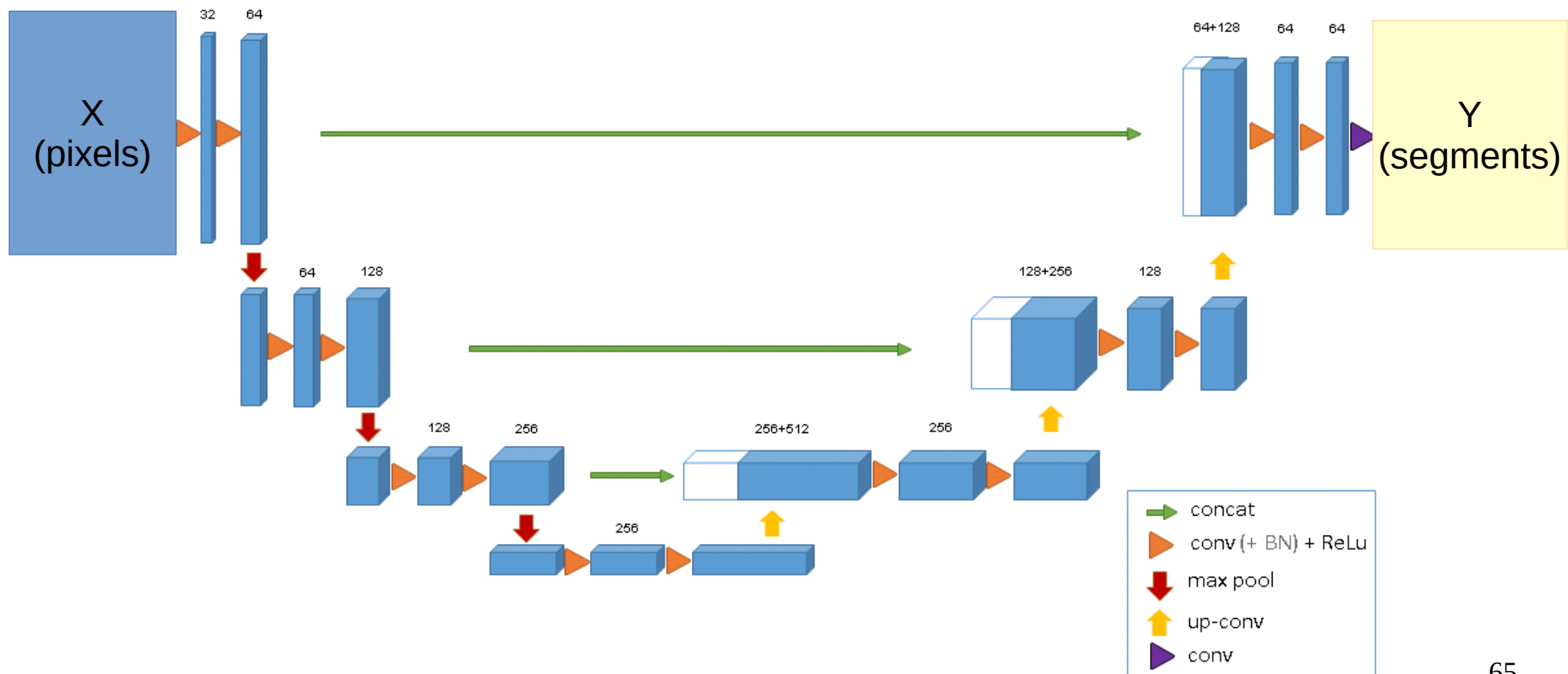
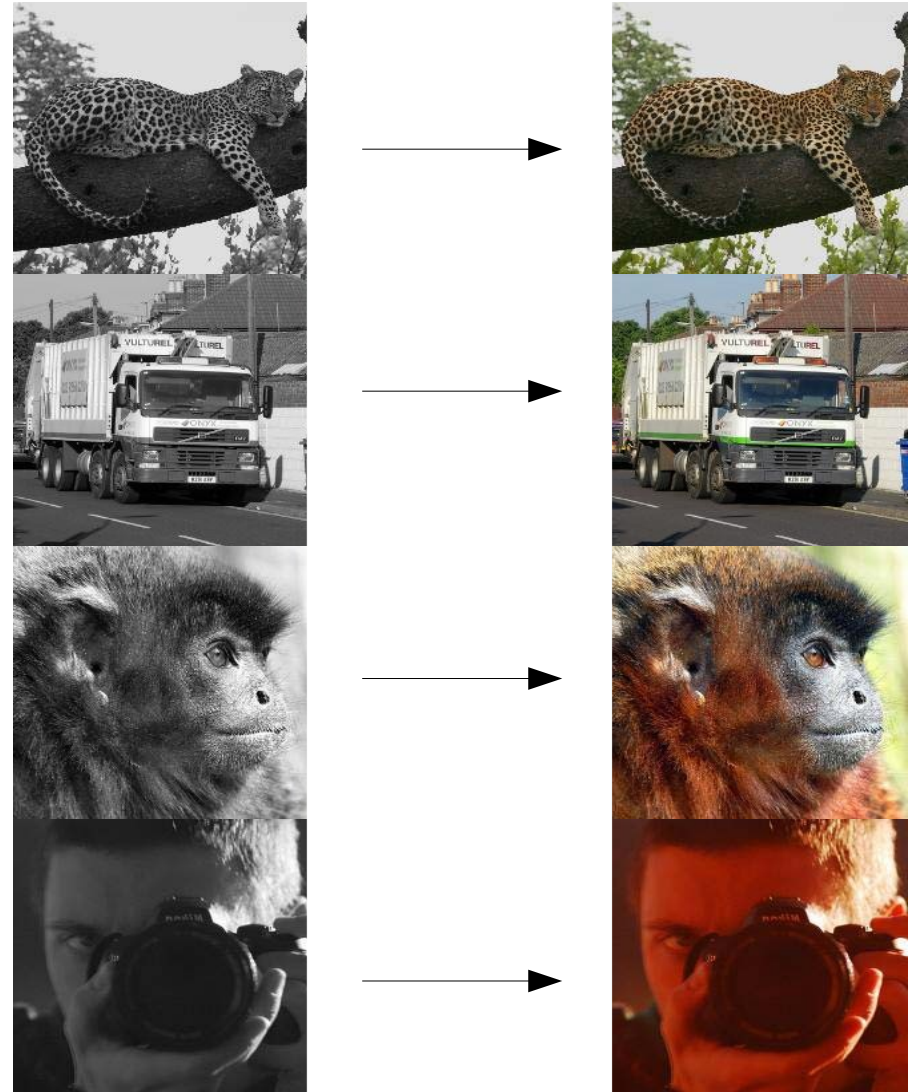


Image coloring

- Input:
 - gray/sepia image
 - mb neighboring frames
- Output:
 - RGB image
 - Same size as input
- Ideas?
 - Data?
 - Loss?
 - Architecture?



Dataset

Image coloring: examples

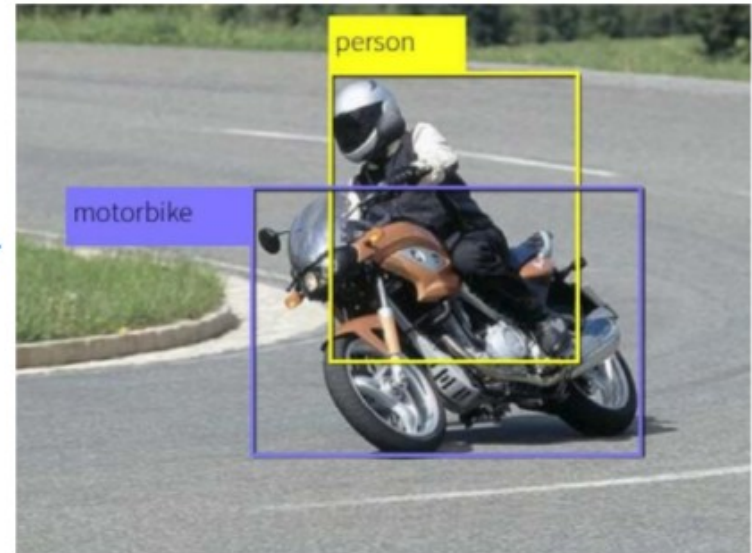


Input/prediction/reference

Bounding Box Regression



Input image

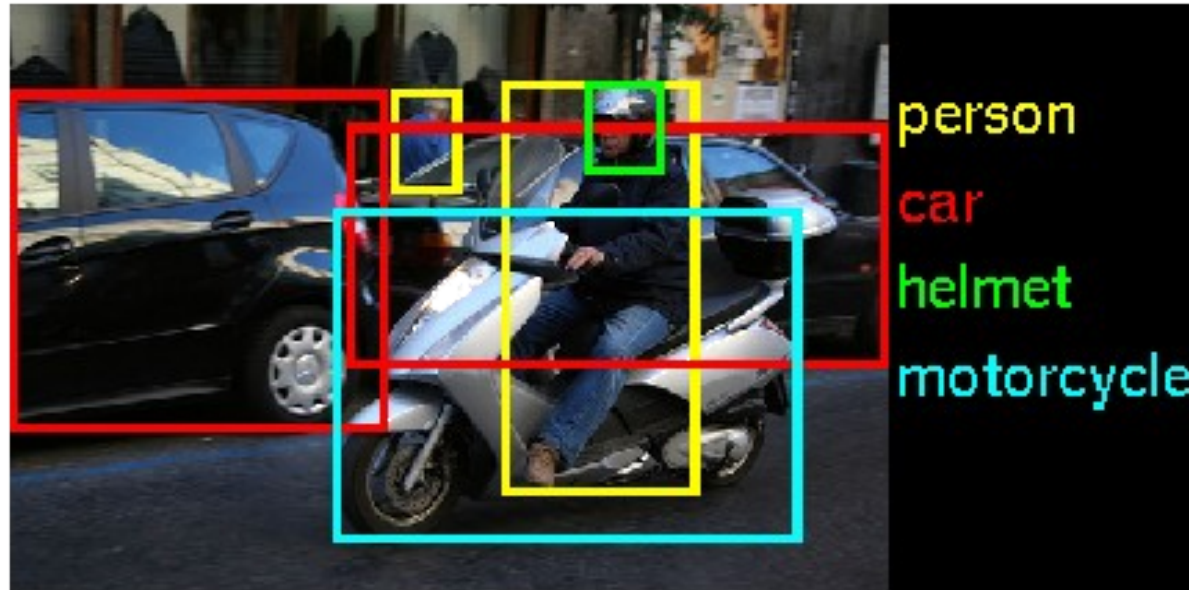


Desired output

- Predict bounding rectangle for the object
- How do we deal with a single square object out of 100 possible objects?

Ideas?

Bounding Box Regression

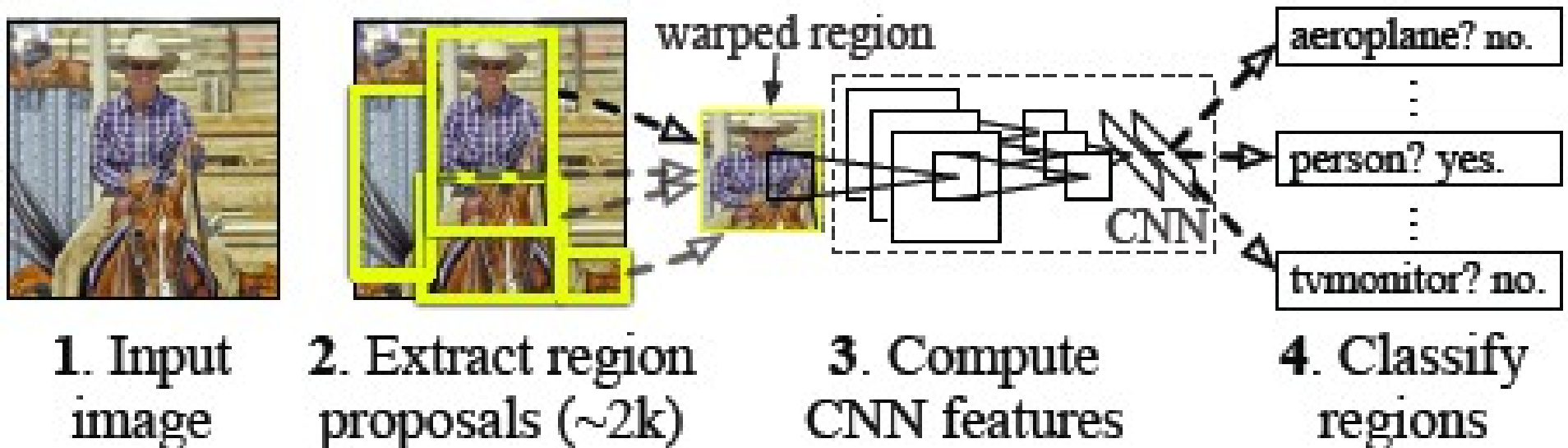


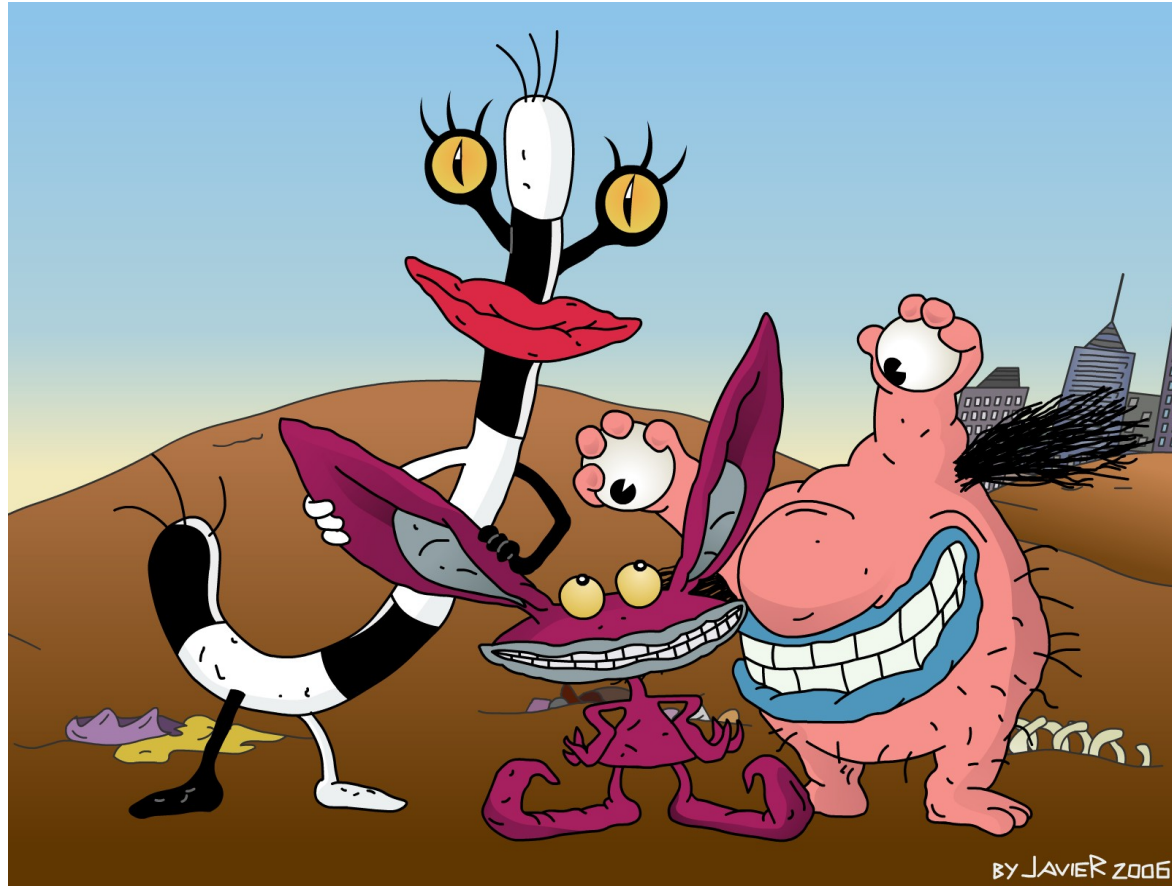
- Multiple squares?
- Non-square shape?
- Arbitrary rotation?

RCNN

- Generate candidates
- Adjust region shapes
- Apply classifier to each region

R-CNN: *Regions with CNN features*





Model zoo in today's seminar